

Fundamentals of Computing and Programming

Dr. Veena Gadad
*Associate Professor,
Department of Computer Science and Engineering,
RV College of Engineering, Bangalore*

Course: Fundamentals of Computing and Programming.

Faculty: Dr. Veena Gadad

Syllabus:

BASICS:

- i) Elementary complexity motivation, concrete complexity, big O notation.
- ii) Integer representation, swtisions, values and variables, types, lvalue, rvalue, unary, binary, ternary operations.
- iii) Numerical errors due to data representations and machine precision. Approximation and error analysis. Illustration through examples.
- iv) Linux tools. Introduction to shell programming.

COMPUTING:

- i) Fundamentals of Computing, Historical perspective, Early computers. Computing machine. Problems, Pseudo-code and flowcharts. Memory, Variables, Values, Instructions, Programs.

INTRODUCTION TO C:

- i) The language of C: Phases of developing a running computer program in C
- ii) Data concepts in C: Constants, Variables, Expressions, Operators, and operator precedence in C
- iii) Statements: Declarations, Input-Output Statements, Compound statements, Selection Statements. Conditions, Logical operators, Precedences. Repetitive statements, While construct, Do-while Construct, For construct.
- iv) Conditionals, if-then, if-then-else, nested conditionals, switch-case. Loops, for, while, repeat, loop invariants, precondition, postcondition.
- v) Data types, size and values. Char, Unsigned and Signed data types. Number systems and representations. Constants, Overflow.
- vi) Arrays. Strings. Multidimensional arrays and matrices.

FUNCTIONS, RECURSIONS, SORTING AND SEARCH:

- i) The prototype declaration, Function definition
- ii) Function call: Passing arguments to a function, by value, by reference. Scope of variable

names. Recursive function calls, Tail recursion. Analysing recursion, Tree of recursion, linear recursion

iii) Sorting problem: Selection Sort, Insertion Sort, Comparison between sorting algorithms. Sorting in multidimensional arrays. Sorting in strings.

IV) Search problem: Linear search and binary search. Comparison between search procedures. Recursive and Iterative formulations.

DATA TYPES IN C:

i) Pointers: Pointer variables. Declaring and dereferencing pointer variables. Pointer Arithmetic. Examples. Accessing arrays through pointers. Pointer types, Pointers and strings. String operations in C.

ii) Structures in C: Motivation, examples, declaration, and use. Operations on structures. Passing structures as function arguments. type defining structures.

iii) Self-referential structures. Dynamic Data Structures. Linked Lists. Examples

iv) File input-output in C. Streams. Input, output and error streams. Opening, closing and reading from files. Programming for command line arguments

Reference Texts:

(a) O J Dahl, E W Dijkstra, C A R Hoare: Structured Programming

(b) David Gries: The Science of Programming

(c) E W Dijkstra: A Short Introduction to the Art of Programming

(d) Dromey: How to solve it by Computer

(e) Goodrich: Data Structures and Algorithms in Java

(f) Thomas A Standish: Data Structures in Java.

Table of Contents

1	Basics	1
1.1	Elementary Complexity Motivation, Concrete Complexity, and Big-O Notation	1
1.1.1	Concrete Complexity	1
1.1.2	From Concrete Complexity to Asymptotic (Big-O) Notation	2
	Common Growth Rates	2
1.1.3	Related Notations	3
	Assignments	3
1.2	Integer Representation, Values, Variables, Types, and Operators	3
1.2.1	Integer Representation in Memory	3
	Bits, Bytes, and Words	4
	Number Base Conversion	4
	Storage Sizes of Integer Types	4
	Unsigned Representation	5
	Signed Representation: Three Historical Schemes	5
	Endianness: Byte Ordering in Memory	5
	Integer Overflow	6
1.2.2	Values, Variables, and Types	6
1.2.3	lvalue and rvalue	6
1.2.4	Unary, Binary, and Ternary Operations	7
	Assignments	7
1.3	Numerical Errors, Machine Precision, and Approximation	8
1.3.1	Motivation	8
1.3.2	Floating-Point Representation (IEEE 754)	8
1.3.3	Machine Epsilon	8
1.3.4	Types of Numerical Error	8
1.3.5	Illustrative Examples	9
	Assignments	9
1.4	Linux Tools and Introduction to Shell Programming	10
1.4.1	Why Linux Tools?	10
1.4.2	Essential Command-Line Tools	10
1.4.3	Introduction to Shell Programming	10
	Shell Programming Constructs	11
1.4.4	Making a Script Executable	11
	Assignments	11
	References	12
2	Computing	13
2.1	Fundamentals of Computing	13
2.1.1	Historical Perspective and Early Computers	13
2.1.2	The Computing Machine, Problems, and Algorithms	14
2.1.3	Pseudocode and Flowcharts	14
2.1.4	Memory, Variables, Values, Instructions, and Programs	17

	Assignments	17
2.2	Introduction to C	18
2.2.1	The Language of C: Phases of Developing a Running Program	18
2.2.2	Data Concepts in C: Constants, Variables, Expressions, Operators, and Precedence	19
	Constants	19
	Variables	19
	Expressions	19
	Operators and Precedence	19
2.2.3	Statements	20
	Declarations	20
	Input-Output Statements	20
	Compound Statements	20
	Selection Statements: Conditions, Logical Operators, Precedence	20
	Repetitive Statements	20
	Assignments	21
2.3	Conditionals and Loops	21
2.3.1	Conditional Statements	21
	if, if-else, and Nested Conditionals	21
	switch-case	21
2.3.2	Loops: for, while, do-while (“repeat”)	21
2.3.3	Loop Invariants, Preconditions, and Postconditions	22
	Assignments	23
2.4	Data Types, Size and Values	23
2.4.1	char, Signed, and Unsigned Types	23
2.4.2	Number Systems and Representations	24
2.4.3	Constants and Overflow, Revisited in C	24
	Assignments	24
2.5	Arrays, Strings, and Multidimensional Arrays	24
2.5.1	One-Dimensional Arrays	24
2.5.2	Strings as Character Arrays	25
2.5.3	Multidimensional Arrays and Matrices	25
	Assignments	25
	References and Notes on Coverage	26
3	Functions, Recursions, Sorting and Search	27
3.1	Functions: Prototypes and Definitions	27
3.1.1	Why Functions?	27
3.1.2	Function Prototype Declaration	27
3.1.3	Function Definition	27
	Assignments	28
3.2	Function Calls: Arguments, Scope, and Recursion	28
3.2.1	Passing Arguments by Value	28
3.2.2	Passing Arguments by Reference (via Pointers)	29
3.2.3	Scope of Variable Names	29
3.2.4	Recursive Function Calls	30
	Tail Recursion	30
	Analysing Recursion: Linear Recursion and the Tree of Recursion	31
	Reasoning About Recursive Correctness	31
	Assignments	31
3.3	Sorting Algorithms	32
3.3.1	The Sorting Problem	32
3.3.2	Selection Sort	32
3.3.3	Insertion Sort	33
3.3.4	Comparison Between Sorting Algorithms	33

3.3.5	Sorting in Multidimensional Arrays	33
3.3.6	Sorting Strings	34
	Assignments	35
3.4	Searching Algorithms	35
3.4.1	The Search Problem	35
3.4.2	Linear Search	36
3.4.3	Binary Search	36
3.4.4	Comparison Between Search Procedures	37
3.4.5	Recursive vs. Iterative Formulations	37
	Assignments	38
	References and Notes on Coverage	38
4	Datatypes in C	39
4.1	Pointers	39
4.1.1	Pointer Variables: Declaration and Dereferencing	39
4.1.2	Pointer Arithmetic	40
4.1.3	Accessing Arrays Through Pointers	40
4.1.4	Pointer Types	40
4.1.5	Pointers and Strings; String Operations in C	41
	Assignments	42
4.2	Structures in C	42
4.2.1	Motivation	42
4.2.2	Declaration and Use	42
4.2.3	Operations on Structures	43
4.2.4	Passing Structures as Function Arguments	43
4.2.5	Type-Defining Structures with <code>typedef</code>	44
	Assignments	44
4.3	Self-Referential Structures and Dynamic Data Structures	44
4.3.1	Self-Referential Structures	44
4.3.2	Dynamic Memory Allocation	45
4.3.3	Linked Lists	45
	Assignments	46
4.4	File Input-Output in C	47
4.4.1	Streams: Input, Output, and Error	47
4.4.2	Opening, Closing, and Reading Files	47
4.4.3	Programming for Command-Line Arguments	48
	Assignments	49
	References and Notes on Coverage	49
5	Previous Year Question Paper with Solutions	51
5.0.1	Midterm	51

Chapter 1

Basics

1. Elementary complexity motivation, concrete complexity, big O notation.
2. Integer representation, swtisions, values and variables, types, lvalue, rvalue, unary, binary, ternary operations.
3. Numerical errors due to data representations and machine precision. Approximation and error analysis. Illustration through examples.
4. Linux tools. Introduction to shell programming.

1.1 Elementary Complexity Motivation, Concrete Complexity, and Big-O Notation

Before writing any program, a natural question is: *how much time and memory will this program need, and how does this scale as the input grows?* This question matters because a program that works perfectly on a small test case may become unusably slow or run out of memory when the input size increases from hundreds to millions of elements.

Consider two algorithms that solve the same problem: searching for an element in a list of n items.

- **Linear search** inspects each element one at a time. In the worst case it performs n comparisons.
- **Binary search** (on a sorted list) repeatedly halves the search space. In the worst case it performs approximately $\log_2 n$ comparisons.

For $n = 10^6$, linear search may require up to one million comparisons, while binary search requires only about 20. This enormous difference is the motivation for studying complexity: it lets us predict and compare the *scalability* of algorithms *before* we implement and run them, rather than discovering performance problems only after deployment (Dromey, *How to Solve it by Computer*, discusses this style of comparative reasoning extensively when contrasting alternative algorithms for the same problem).

1.1.1 Concrete Complexity

[Concrete Complexity] The *concrete complexity* (also called *exact* or *operation count* complexity) of an algorithm is the precise count, as a function of input size n , of the number of “basic operations” (comparisons, additions, assignments, etc.) the algorithm performs.

Concrete complexity gives an exact expression, e.g. $T(n) = 3n^2 + 5n + 2$, rather than a growth-rate classification. This idea, and the terminology, follows the treatment in Goodrich’s *Data Structures and Algorithms* and is a precursor to asymptotic (big-O) analysis: we first count operations exactly, and only later abstract away constant factors and lower-order terms.

[Counting Operations] Consider the following pseudocode that sums the elements of an array of size n :

Algorithm 1 ArraySum

```

1:  $sum \leftarrow 0$  ▷ 1 assignment
2: for  $i \leftarrow 0$  to  $n - 1$  do ▷  $n$  iterations
3:    $sum \leftarrow sum + A[i]$  ▷ 1 addition + 1 assignment per iteration
4: end for
5: return  $sum$ 

```

The concrete complexity is $T(n) = 1 + 2n$ operations (one initialization, plus two operations per loop iteration).

1.1.2 From Concrete Complexity to Asymptotic (Big-O) Notation

Exact operation counts are useful, but they are sensitive to implementation details (choice of basic operation, compiler optimizations, hardware) that do not reflect the *fundamental* growth behaviour of the algorithm. *Asymptotic analysis* abstracts away constants and lower-order terms, focusing on how running time grows as $n \rightarrow \infty$.

[Big-O Notation] Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$. We say $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

Informally, $f(n) = O(g(n))$ means f grows no faster than g , up to a constant factor, for large enough n .

For $T(n) = 3n^2 + 5n + 2$, we have $T(n) = O(n^2)$, since for $n \geq 1$,

$$3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2.$$

Here $c = 10$ and $n_0 = 1$ satisfy the definition.

Common Growth Rates

Notation		Name		
$O(1)$		Constant		
$O(\log n)$		Logarithmic		
$O(n)$		Linear		
$O(n \log n)$		Linearithmic		
$O(n^2)$		Quadratic		
$O(n^3)$		Cubic		
$O(2^n)$		Exponential		

n	$\log_2 n$	n	$n \log_2 n$	n^2
10	3.3	10	33	100
100	6.6	100	664	10,000
1000	10.0	1000	9966	1,000,000

This table illustrates why, for large n , an $O(n \log n)$ algorithm (e.g. merge sort) drastically outperforms an $O(n^2)$ algorithm (e.g. bubble sort), even though for small n the difference may be negligible or even reversed due to constant factors.

[Concrete Complexity of Bubble Sort] Bubble sort on an array of size n uses a nested loop: the outer loop runs $n - 1$ times, and for outer-loop iteration i (starting at 1), the inner loop performs $n - i$ comparisons. The total number of comparisons is

$$T(n) = \sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \cdots + 1 = \frac{n(n - 1)}{2}.$$

For $n = 5$: $T(5) = \frac{5 \cdot 4}{2} = 10$ comparisons. For $n = 10$: $T(10) = \frac{10 \cdot 9}{2} = 45$ comparisons. For $n = 1000$: $T(1000) = \frac{1000 \cdot 999}{2} = 499,500$ comparisons – illustrating both the exact quadratic formula and why we simplify it to $O(n^2)$ for large n .

[Numeric Comparison: Linear vs. Binary Search] For an array of $n = 1,048,576$ ($= 2^{20}$) sorted elements, linear search may require up to 1,048,576 comparisons in the worst case, while binary search requires at most $\log_2(1,048,576) = 20$ comparisons – a factor of over 50,000 $\times$ fewer operations, making the abstract growth-rate distinction concrete and dramatic.

1.1.3 Related Notations

- $\Omega(g(n))$: lower bound – $f(n)$ grows *at least* as fast as $g(n)$.
- $\Theta(g(n))$: tight bound – $f(n)$ grows *exactly* as fast as $g(n)$ (both O and Ω hold).

[Board Illustration] Show that $f(n) = 2n^2 + 3n$ is $\Theta(n^2)$:

$$\begin{aligned} 2n^2 + 3n &\leq 5n^2 \quad \text{for } n \geq 1 &\Rightarrow f(n) &= O(n^2) \\ 2n^2 + 3n &\geq 2n^2 \quad \text{for all } n \geq 0 &\Rightarrow f(n) &= \Omega(n^2) \end{aligned}$$

Hence $f(n) = \Theta(n^2)$.

Teaching tip: work through linear search vs. binary search, and bubble sort vs. merge sort, deriving concrete complexity first, then simplifying to big-O, so students see the connection between exact counting and asymptotic classification.

Assignments

1. Count the exact number of comparisons and assignments performed by linear search on an array of size $n = 20$, in the worst case and in the best case.
2. Derive the concrete complexity $T(n)$ of a nested loop that, for each i from 1 to n , iterates j from 1 to i and performs one addition. Simplify your expression for $T(n)$ using summation formulas, and evaluate $T(6)$ numerically.
3. Prove, using the definition of big-O, that $5n^3 + 2n^2 + 100 = O(n^3)$. State the constants c and n_0 you used, and verify your inequality numerically at $n = 4$.
4. Arrange the following functions in increasing order of growth rate: n^2 , $\log n$, $n \log n$, 2^n , n , 1. Then compute the numeric value of each function at $n = 16$ and confirm your ordering.
5. An algorithm takes 2 milliseconds for $n = 200$. If its running time is $\Theta(n^2)$, estimate its running time for $n = 2000$. Repeat assuming instead that it is $\Theta(n \log n)$.
6. Using the bubble sort formula $T(n) = n(n-1)/2$, compute the exact number of comparisons for $n = 50$, and compare it against the big-O estimate $O(n^2)$ evaluated as $50^2 = 2500$.

1.2 Integer Representation, Values, Variables, Types, and Operators

1.2.1 Integer Representation in Memory

Computers store all data as sequences of bits. An integer type of width k bits can represent 2^k distinct values. This subsection develops the idea from the ground up: bits and bytes, number-base conversion, storage widths, unsigned and signed encodings, byte ordering, and the pitfalls of overflow.

Bits, Bytes, and Words

- A **bit** is the smallest unit of storage, holding a single binary digit, 0 or 1.
- A **byte** is a group of 8 bits, and is the smallest *addressable* unit of memory on virtually all modern systems – i.e. every memory address refers to one byte.
- A **word** is the natural unit of data a processor’s registers and data paths operate on – commonly 4 bytes (32 bits) or 8 bytes (64 bits) on modern architectures.
- A **nibble** is half a byte, i.e. 4 bits, and conveniently corresponds to exactly one hexadecimal digit.

An integer occupying k bits can distinguish 2^k distinct bit patterns; how those patterns are *interpreted* as numeric values (unsigned or signed, and by which encoding) is a separate decision from how many bits are used to store them.

Number Base Conversion

Programmers routinely work with binary, octal, decimal, and hexadecimal representations of the same integer value. The positional value of a digit b at position i (counting from 0 at the rightmost digit) in base r is $b \cdot r^i$.

[Binary to Decimal] $(10110)_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 16 + 0 + 4 + 2 + 0 = 22$.

[Decimal to Binary via Repeated Division] To convert 22 to binary, repeatedly divide by 2 and record remainders:

$$22 \div 2 = 11 \text{ r } 0, \quad 11 \div 2 = 5 \text{ r } 1, \quad 5 \div 2 = 2 \text{ r } 1, \quad 2 \div 2 = 1 \text{ r } 0, \quad 1 \div 2 = 0 \text{ r } 1.$$

Reading remainders bottom-to-top gives $22 = (10110)_2$, matching the previous example.

[Hexadecimal Shorthand] Because $16 = 2^4$, each hex digit corresponds to exactly one nibble, making hex a compact way to write bit patterns. The byte 10110110 splits into nibbles 1011 and 0110, i.e. B and 6, so the byte is written 0xB6 in code. This is why memory dumps, addresses, and bit masks are almost always shown in hexadecimal rather than binary or decimal.

Decimal	Binary	Octal	Hex
0	0000	0	0x0
5	0101	5	0x5
10	1010	12	0xA
15	1111	17	0xF

Storage Sizes of Integer Types

The number of bits allotted to an integer type fixes both its range and its memory footprint. Typical widths on common 64-bit systems (sizes are implementation-defined and can vary by platform/compiler):

Type	Width (bits)	Unsigned range	Signed range (two’s complement)
char	8	[0, 255]	[−128, 127]
short	16	[0, 65,535]	[−32,768, 32,767]
int	32	[0, 4,294,967,295]	[−2,147,483,648, 2,147,483,647]
long long	64	[0, $2^{64} - 1$]	[− 2^{63} , $2^{63} - 1$]

A useful rule of thumb to derive on the board: a k -bit unsigned type holds 2^k values, 0 through $2^k - 1$; a k -bit signed (two’s complement) type holds the same 2^k values, but split asymmetrically across zero as -2^{k-1} through $2^{k-1} - 1$ (one extra negative value, since there is no “−0” pattern to waste, as explained below).

Unsigned Representation

An unsigned k -bit integer represents values in the range $[0, 2^k - 1]$ directly as a binary number:

$$v = \sum_{i=0}^{k-1} b_i \cdot 2^i.$$

[Characters Are Stored as Integers] A `char` is really just an 8-bit integer interpreted using a character encoding such as ASCII. The character `'A'` is stored as the unsigned integer 65 (binary 01000001), `'a'` as 97 (01100001), and `'0'` as 48 (00110000). This is why, in C, `'a' - 'A'` evaluates to 32, and why `'5' - '0'` evaluates to the integer 5 – both are ordinary integer subtractions on the underlying numeric codes.

[Unsigned Range Check] For a 16-bit unsigned type, the range is $[0, 2^{16} - 1] = [0, 65,535]$. The value 70,000 cannot be stored; it would overflow (wrap around) to $70,000 - 65,536 = 4,464$.

Signed Representation: Three Historical Schemes

Representing negative numbers requires reserving some encoding for the sign. Three schemes are worth contrasting on the board, since seeing why two's complement *wins* clarifies why it is universal in modern hardware.

(a) Sign-magnitude. The most significant bit is a pure sign flag (0 = positive, 1 = negative); the remaining $k - 1$ bits store the magnitude. For $k = 8$: $+5 = 00000101$, $-5 = 10000101$. *Drawback:* two representations of zero (00000000 and 10000000), and addition circuitry must special-case signs.

(b) One's complement. Negation is performed by inverting every bit. For $k = 8$: $+5 = 00000101$, $-5 = 11111010$. *Drawback:* still has two representations of zero (00000000 and 11111111), and addition requires an “end-around carry” correction.

(c) Two's complement (used in practice). Negation is performed by inverting all bits and adding 1:

$$v = -b_{k-1} \cdot 2^{k-1} + \sum_{i=0}^{k-2} b_i \cdot 2^i,$$

where b_{k-1} is the most significant (sign) bit. *Advantages that make it the universal choice:* there is a single representation of zero (00000000); ordinary binary addition circuitry works unmodified for both signed and unsigned operands (no special-casing of the sign bit); and the range is $[-2^{k-1}, 2^{k-1} - 1]$ – one extra negative value compared to the positive range, precisely because no bit pattern is wasted on a second zero.

An 8-bit two's complement representation of -5 is obtained by:

1. Write 5 in binary: 00000101.
2. Invert all bits (one's complement): 11111010.
3. Add 1: 11111011.

So -5 is represented as 11111011. Check: $-1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -128 + 123 = -5$. ✓

Endianness: Byte Ordering in Memory

A multi-byte integer (e.g. a 32-bit `int`) occupies several consecutive memory addresses, and different architectures order its bytes differently:

- **Big-endian:** the most significant byte is stored at the lowest memory address (the “natural” reading order, left to right).

- **Little-endian:** the least significant byte is stored at the lowest memory address (used by x86/x86-64 and most ARM configurations).

The 32-bit value 0x12345678 stored at address 0x1000 appears in memory as:

Address	0x1000	0x1001	0x1002	0x1003
Big-endian	12	34	56	78
Little-endian	78	56	34	12

Endianness matters whenever raw bytes cross machine boundaries – e.g. network protocols, binary file formats, or reinterpreting a block of memory via pointer casts – and is a common source of subtle bugs when such data is exchanged between systems with different byte orders.

Integer Overflow

When an operation produces a value outside the representable range of a type, *overflow* occurs, and the result silently “wraps around” rather than raising an error in most C-family languages.

[Signed Overflow] Adding 1 to the maximum 8-bit signed value 127 (01111111) yields 10000000 in ordinary binary addition, which is the two’s-complement encoding of -128 – so $127 + 1$ “wraps around” to -128 instead of producing 128. This is a classic wrap-around bug that students should be warned about explicitly, since signed overflow is *undefined behaviour* in C/C++ and can be silently miscompiled.

[Unsigned Overflow] For an 8-bit unsigned type, the maximum value is 255 (11111111). Adding 1 gives 100000000, which truncates to 8 bits as 00000000, i.e. 0. Unlike signed overflow, unsigned overflow is well-defined (modulo 2^k “wraparound” arithmetic) in languages such as C, and is sometimes used deliberately, e.g. in hash functions or circular counters.

Board exercise: ask students to predict the output of `unsigned char c = 250; c += 10;` and verify by hand using modulo-256 arithmetic ($260 \bmod 256 = 4$).

1.2.2 Values, Variables, and Types

[Variable] A *variable* is a named storage location in memory that holds a *value*. A variable has an associated *type*, which determines the set of values it may hold and the operations that may be performed on it.

[Type] A *type* is a classification specifying: (i) the size (in bits/bytes) of storage required, (ii) the interpretation of the stored bit pattern (e.g. integer, floating point, character), and (iii) the set of valid operations.

Common primitive types include `int`, `char`, `float`, `double`, and `boolean`, each with a fixed size and interpretation as covered in the introductory chapters of Goodrich and Standish.

1.2.3 lvalue and rvalue

[lvalue] An *lvalue* (“locator value”) refers to an expression that identifies a persistent memory location, i.e., something that can legally appear on the *left-hand side* of an assignment.

[rvalue] An *rvalue* (“right value”) refers to a temporary value that does not necessarily persist in an addressable memory location; it may only appear on the *right-hand side* of an assignment.

```

1 int x = 5;           // x is an lvalue; 5 is an rvalue
2 int y = x + 1;       // x + 1 is an rvalue (a temporary result)
3 x = y;              // valid: x is lvalue, y is rvalue
4 10 = x;              // INVALID: 10 is not an lvalue

```

The distinction is essential for understanding compiler error messages such as “lvalue required as left operand of assignment.”

1.2.4 Unary, Binary, and Ternary Operations

[Arity of an Operator] The *arity* of an operator is the number of operands it takes.

- **Unary operators** take a single operand: negation $-x$, logical NOT $!x$, increment $++x$, decrement $--x$.
- **Binary operators** take two operands: arithmetic ($+$, $-$, $*$, $/$, $\%$), relational ($=$, $<$), logical ($\&\&$, $||$).
- **Ternary operator** takes three operands: the conditional expression `condition ? expr1 : expr2`, which is the only ternary operator in most C-family languages.

```
1 int a = -5;           // unary minus
2 int b = a + 10;       // binary addition
3 int max = (a > b) ? a : b; // ternary conditional operator
```

The ternary expression above is equivalent to, but more compact than:

```
1 int max;
2 if (a > b) { max = a; } else { max = b; }
```

[Bitwise Operators on Concrete Values] Bitwise operators act on the individual bits of an integer's representation. Let $a = 12$ (1100) and $b = 10$ (1010), using 4-bit patterns for clarity:

$$a \& b = 1000 = 8, \quad a | b = 1110 = 14, \quad a \oplus b = 0110 = 6, \quad a \ll 1 = 11000 = 24.$$

Here $\&$ is bitwise AND, $|$ is bitwise OR, $\wedge$ is bitwise XOR, and $\ll$ is a left shift (equivalent to multiplying by 2 per shifted position, here $12 \times 2 = 24$).

Teaching tip: draw a memory diagram on the board showing a labelled box (the variable/lvalue), its type-determined size, and the bit pattern stored inside, and contrast it with an rvalue that exists only transiently on the evaluation stack.

Assignments

1. Convert $(173)_{10}$ to binary, octal, and hexadecimal, showing your working.
2. Find the 8-bit two's complement representation of -42 . Verify your answer by converting it back to decimal.
3. Given the 8-bit pattern 11010011, find its value if interpreted as (a) unsigned, (b) sign-magnitude, (c) one's complement, (d) two's complement.
4. Show how the 16-bit value 0xABCD would be laid out in memory at addresses 0x2000–0x2001 under big-endian and little-endian ordering.
5. For an 8-bit unsigned `char`, compute the result of $250 + 10$ using modulo-256 arithmetic, and explain why this differs from ordinary integer addition.
6. Classify each of the following as unary, binary, or ternary, and identify which are lvalues vs. rvalues: $-x$, $a + b$, $(x > 0) ? 1 : -1$, $x = 5$, $++count$.
7. Let $x = 27$ and $y = 6$ (as 8-bit unsigned integers). Compute $x \& y$, $x | y$, $x \oplus y$, and $x \ll 2$, showing the binary representation at each step.
8. A 16-bit signed integer currently holds the maximum value 32,767. What value results from adding 5 to it, assuming two's-complement wraparound? Show the binary addition.
9. If 'A' is stored as the integer 65, what integer is used to store 'F'? What character corresponds to the integer 122?

1.3 Numerical Errors, Machine Precision, and Approximation

1.3.1 Motivation

Computers represent real numbers using a *finite* number of bits, but the set of real numbers is infinite and continuous. Consequently, most real numbers cannot be represented exactly, and arithmetic operations on their approximations introduce errors. Understanding these errors is essential for writing numerically reliable programs – this is the central theme of the numerical chapters in Dromey’s *How to Solve it by Computer*.

1.3.2 Floating-Point Representation (IEEE 754)

A floating-point number is represented as:

$$x = (-1)^s \times 1.m \times 2^{e-\text{bias}}$$

where s is the sign bit, m is the mantissa (fraction), and e is the biased exponent. The IEEE 754 single-precision (32-bit) format uses 1 sign bit, 8 exponent bits, and 23 mantissa bits; double precision (64-bit) uses 1, 11, and 52 bits respectively.

Because the mantissa has finite length, only finitely many values can be represented exactly, and most real numbers are rounded to the *nearest representable value*.

1.3.3 Machine Epsilon

[Machine Epsilon] Machine epsilon, $\varepsilon_{\text{mach}}$, is the smallest positive number such that $1 + \varepsilon_{\text{mach}} \neq 1$ in floating-point arithmetic. It characterizes the resolution, or granularity, of the floating-point number system near 1.0.

For IEEE 754 double precision, $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

In many languages, evaluating `0.1 + 0.2 == 0.3` returns `false`, because 0.1 and 0.2 cannot be represented exactly in binary floating point; their sum is approximately 0.30000000000000004, not exactly 0.3.

1.3.4 Types of Numerical Error

- **Round-off error:** the difference between a real number and its nearest floating-point representation, arising because of finite precision.
- **Truncation error:** the error introduced by approximating an infinite or continuous process (e.g. an infinite series or a derivative) with a finite computation.
- **Absolute error:** $|x_{\text{true}} - x_{\text{approx}}|$.
- **Relative error:** $\frac{|x_{\text{true}} - x_{\text{approx}}|}{|x_{\text{true}}|}$, which is often more meaningful since it accounts for the magnitude of the quantity.

[Absolute vs. Relative Error] Suppose $\sqrt{2} = 1.41421356\dots$ is approximated as 1.414. The absolute error is

$$|1.41421356 - 1.414| = 0.00021356 \approx 2.14 \times 10^{-4},$$

and the relative error is

$$\frac{0.00021356}{1.41421356} \approx 1.51 \times 10^{-4} = 0.0151\%.$$

Now compare approximating a much larger number, $\sqrt{20000} = 141.4213562\dots$, by 141.421: the absolute error is about 3.6×10^{-4} (similar magnitude), but the relative error is only about 2.5×10^{-6} – illustrating why relative error, not absolute error, is the meaningful measure when comparing approximation quality across different scales.

1.3.5 Illustrative Examples

[Catastrophic Cancellation] Consider computing $f(x) = \frac{1 - \cos x}{x^2}$ for small x (e.g. $x = 10^{-8}$). Mathematically, $f(x) \rightarrow \frac{1}{2}$ as $x \rightarrow 0$. However, in floating point, $\cos x \approx 1$ for very small x , so $1 - \cos x$ subtracts two nearly equal numbers, losing most of the significant digits (*catastrophic cancellation*), producing a wildly inaccurate result unless the expression is rewritten in a numerically stable form, e.g. using $1 - \cos x = 2 \sin^2(x/2)$.

[Truncation Error in Series Approximation] The Taylor series for e^x is

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!}.$$

A program that sums only the first N terms incurs a truncation error equal to the magnitude of the omitted tail $\sum_{k=N+1}^{\infty} x^k/k!$. Increasing N reduces truncation error but does not eliminate round-off error from the finite-precision arithmetic used in each partial sum.

[Taylor Truncation for e^1 – Worked Numerically] The true value is $e = 2.718281828\dots$. Using only $N = 4$ terms ($k = 0, 1, 2, 3$):

$$S_4 = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} = 1 + 1 + 0.5 + 0.1\bar{6} = 2.6\bar{6}.$$

The truncation error is $|e - S_4| = |2.718281828 - 2.666667| \approx 0.0516$. Using $N = 6$ terms (up to $k = 5$):

$$S_6 = 2.6\bar{6} + \frac{1}{4!} + \frac{1}{5!} = 2.6\bar{6} + 0.041\bar{6} + 0.008\bar{3} = 2.71\bar{6},$$

giving truncation error $|e - S_6| \approx 0.0016$ – roughly $32\times$ smaller than with $N = 4$, demonstrating how rapidly the Taylor truncation error shrinks as more terms are included.

[Accumulated Round-off Error] Summing 10^7 copies of 0.1 in single-precision floating point does not yield exactly 10^6 ; small round-off errors accumulate with each addition. This motivates numerically stable summation algorithms such as *Kahan summation*, which tracks and compensates for lost low-order bits.

Teaching tip: have students run a short program (e.g. in C or Python) that sums 0.1 many times and compares the result to the expected value, to give a hands-on demonstration of accumulated floating-point error.

Assignments

1. Explain, in your own words, the difference between round-off error and truncation error, giving one example of each that is not from the notes.
2. Write a short program (in any language) that checks whether $0.1 + 0.2 == 0.3$ and explain the observed output in terms of floating-point representation.
3. Compute the absolute and relative error when approximating $\pi = 3.14159265\dots$ by 3.14 .
4. For small x , explain why computing $f(x) = (1 - \cos x)/x^2$ directly is numerically unstable, and show how rewriting it using $1 - \cos x = 2 \sin^2(x/2)$ avoids the instability.
5. Using the Taylor series $e^x = \sum_{k=0}^{\infty} x^k/k!$, compute the partial sum S_5 (first five terms, $k = 0$ to 4) for $x = 1$, and find the absolute truncation error compared to the true value $e = 2.718281828$.
6. A quantity is measured as $x_{\text{approx}} = 9.97$, while its true value is $x_{\text{true}} = 10.00$. Compute the absolute error and the relative error (as a percentage).
7. The single-precision machine epsilon is approximately $\varepsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$. If a computation involves 1000 sequential additions, each introducing an error on the order of $\varepsilon_{\text{mach}}$, estimate the order of magnitude of the total accumulated round-off error.

1.4 Linux Tools and Introduction to Shell Programming

1.4.1 Why Linux Tools?

Most professional software development, scientific computing, and systems programming occurs on Unix/Linux systems. Fluency with the command line enables automation of repetitive tasks, efficient file manipulation, and the ability to combine small, single-purpose tools into powerful pipelines – a philosophy often summarized as “do one thing well.”

1.4.2 Essential Command-Line Tools

Command	Purpose
<code>pwd</code>	Print current working directory
<code>ls -l</code>	List directory contents (long format)
<code>cd</code>	Change directory
<code>mkdir, rmdir</code>	Create / remove directories
<code>cp, mv, rm</code>	Copy, move/rename, remove files
<code>cat, less</code>	Display file contents
<code>grep</code>	Search text using patterns
<code>find</code>	Search for files by name/attributes
<code>chmod, chown</code>	Change permissions / ownership
<code>man</code>	Display manual pages

[Combining Tools with Pipes]

```
1 # Count the number of lines in all .c files containing "TODO"
2 grep -l "TODO" *.c | wc -l
```

Here the output of `grep -l` (filenames containing a match) is *pipelined* (`|`) as input to `wc -l` (line/word count), illustrating the Unix philosophy of composing small tools.

[Arithmetic in the Shell] Unlike most commands, the shell can perform integer arithmetic directly using `$(())` arithmetic expansion:

```
1 $ a=15
2 $ b=4
3 $ echo $((a + b))      # 19
4 $ echo $((a * b))      # 60
5 $ echo $((a / b))      # 3    (integer division, truncates)
6 $ echo $((a % b))      # 3    (remainder)
```

Note that `$(a / b)` performs *integer* division ($15/4 = 3.75$ truncates to 3), directly paralleling the integer-division behaviour discussed for typed variables in Section 1.2.

1.4.3 Introduction to Shell Programming

A *shell script* is a text file containing a sequence of shell commands, executed by an interpreter such as `bash`.

[A Simple Shell Script]

```
1 #!/bin/bash
2 # backup.sh: back up all .txt files into a timestamped folder
3
4 DEST="backup_$(date +%Y%m%d_%H%M%S)"
5 mkdir "$DEST"
6
7 for file in *.txt; do
8     cp "$file" "$DEST"
9     echo "Copied $file to $DEST"
10 done
```

```

11
12 echo "Backup complete: $DEST"

```

Key concepts illustrated here: the shebang line (`#!/bin/bash`), variable assignment (`DEST=...`), command substitution (`${...}`), a `for` loop, and string interpolation with double quotes.

Shell Programming Constructs

- **Variables:** `name=value` (no spaces around `=`); accessed as `$name`.
- **Conditionals:**

```

1 if [ "$1" -gt 0 ]; then
2     echo "Positive"
3 else
4     echo "Non-positive"
5 fi

```

- **Loops:** `for`, `while`, `until`.

- **Functions:**

```

1 greet() {
2     echo "Hello, $1!"
3 }
4 greet "World"

```

- **Redirection and Pipes:** `>` (overwrite output to file), `>>` (append), `<` (input from file), `|` (pipe output of one command to another).

1.4.4 Making a Script Executable

```

1 chmod +x backup.sh
2 ./backup.sh

```

`chmod +x` grants execute permission; `./` indicates that the script should be run from the current directory.

Teaching tip: run a live terminal demo building up the `backup.sh` script incrementally – start with a bare copy command, then add the loop, then the timestamped directory – so students see how scripts are typically developed iteratively rather than written all at once.

Assignments

1. Write a single shell command using `find` and `grep` to list all `.c` files in the current directory tree that contain the word `main`.
2. Explain the difference between `>` and `>>` redirection, and between `|` (pipe) and `>` (redirection).
3. Write a shell script `countdown.sh` that takes a number `n` as a command-line argument and prints the integers from `n` down to 1, one per line, using a `while` loop. Trace its output by hand for `n = 7`.
4. Modify the `backup.sh` script from the notes so that it only copies files modified in the last 24 hours (hint: look up the `find -mtime` option).
5. What permission does `chmod +x script.sh` grant, and why is it necessary before running `./script.sh`?
6. Write a shell function that takes a filename as an argument and prints “exists” or “not found” depending on whether the file is present in the current directory.

7. Using shell arithmetic expansion `$((...))`, compute the value of $((23 + 7) * 3) \% 5$ and verify it by hand.
8. Write a shell script that uses a `for` loop and `$((...))` to compute and print the sum $1 + 2 + \dots + 10$, and confirm the final printed value is 55.

References

- M. T. Goodrich, R. Tamassia, M. H. Goldwasser. *Data Structures and Algorithms in Java*.
- T. A. Standish. *Data Structures in Java*.
- R. G. Dromey. *How to Solve it by Computer*.

Chapter 2

Computing

1. Fundamentals of Computing, Historical perspective, Early computers. Computing machine. Problems, Pseudo-code and flowcharts. Memory, Variables, Values, Instructions, Programs.
2. INTRODUCTION TO C: The language of C: Phases of developing a running computer program in C
3. Data concepts in C: Constants, Variables, Expressions, Operators, and operator precedence in C
4. Statements: Declarations, Input-Output Statements, Compound statements, Selection Statements. Conditions, Logical operators, Precedences. Repetitive statements, While construct, Do-while Construct, For construct.
5. Conditionals, if-then, if-then-else, nested conditionals, switch-case. Loops, for, while, repeat, loop invariants, precondition, postcondition.
6. Data types, size and values. Char, Unsigned and Signed data types. Number systems and representations. Constants, Overflow.
7. Arrays. Strings. Multidimensional arrays and matrices.

2.1 Fundamentals of Computing

2.1.1 Historical Perspective and Early Computers

Before writing a single line of code, it is useful for students to see *why* computers are built the way they are. The idea of a mechanical device that carries out calculations automatically long predates electronics.

- **Charles Babbage (1830s)** designed the *Analytical Engine*, a mechanical general-purpose computer with a “mill” (processor) and a “store” (memory) – the conceptual ancestor of modern CPU/memory separation. Ada Lovelace wrote what is considered the first algorithm intended for the machine.
- **Alan Turing (1936)** formalized the notion of computation itself with the *Turing machine*, a mathematical model defining what it means for a function to be *computable* – the theoretical foundation underlying every real computer.
- **ENIAC (1945)** was one of the first electronic general-purpose computers, built with vacuum tubes; it was reprogrammed by physically rewiring plugboards, illustrating the historical distinction between *hardware* and what we now call *software*.
- **John von Neumann (1945)** proposed the *stored-program concept*: instructions and data reside together in the same memory, and instructions are fetched and executed sequentially. Virtually every computer today, including the one running this lecture, is a *von Neumann machine*.

Generation	Technology	Approx. Era
1st	Vacuum tubes	1940s–1950s
2nd	Transistors	1950s–1960s
3rd	Integrated circuits	1960s–1970s
4th	Microprocessors (VLSI)	1970s–present

This progression – from room-sized machines programmed by rewiring, to microprocessors small enough to fit in a phone – motivates why programming languages evolved: as hardware became more capable, the need arose for humans to express instructions in ways closer to human thought than to raw wiring diagrams.

2.1.2 The Computing Machine, Problems, and Algorithms

[Computing Machine] A *computing machine* (computer) is a device that accepts input, executes a finite sequence of instructions on that input following a stored program, and produces output. At its core it consists of a processor (executes instructions), memory (stores instructions and data), and input/output devices.

Solving a problem with a computer follows a standard pipeline, a theme developed at length in Dromey’s *How to Solve it by Computer*:

Problem $\longrightarrow$ Algorithm $\longrightarrow$ Program $\longrightarrow$ Execution $\longrightarrow$ Result

[Algorithm] An *algorithm* is a finite, unambiguous sequence of well-defined steps that transforms a given input into a desired output, and that is guaranteed to terminate.

An algorithm is not yet a program: it is a language-independent description of *what* to do, which must later be translated into a specific programming language’s syntax before a machine can execute it.

2.1.3 Pseudocode and Flowcharts

Two common notations are used to express algorithms before committing to actual code: *pseudocode* (structured, language-like text) and *flowcharts* (graphical diagrams).

[Pseudocode: Sum of First N Natural Numbers]

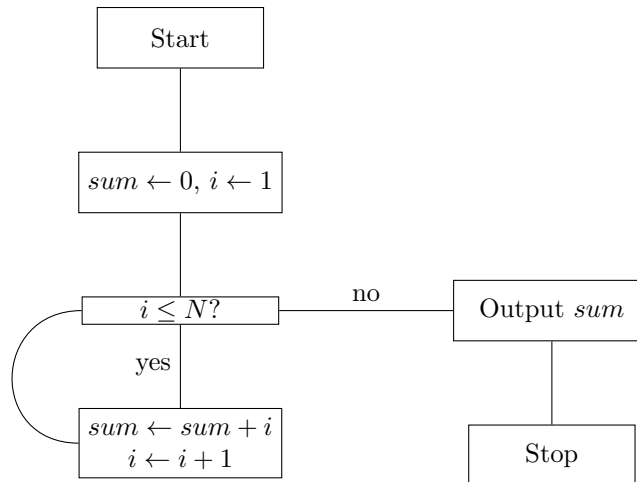
Algorithm 2 SumOfFirstN

```

1: Input: a positive integer  $N$ 
2: Output: the sum  $1 + 2 + \dots + N$ 
3:  $sum \leftarrow 0$ 
4:  $i \leftarrow 1$ 
5: while  $i \leq N$  do
6:    $sum \leftarrow sum + i$ 
7:    $i \leftarrow i + 1$ 
8: end while
9: return  $sum$ 

```

[Flowchart for the Same Algorithm]



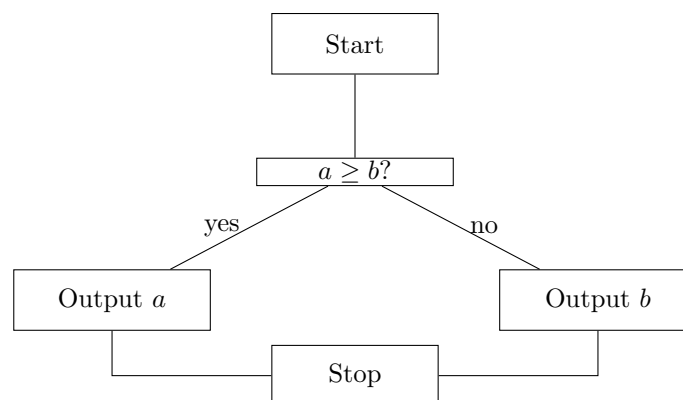
The diamond represents a decision (branch), rectangles represent processing steps, and the loop is drawn as a backward arrow returning to the condition – the graphical counterpart of the **while** loop in the pseudocode above.

[Maximum of Two Numbers – A Pure Selection Algorithm]

Algorithm 3 MaxOfTwo

```

1: Input: two numbers  $a, b$ 
2: Output: the larger of  $a$  and  $b$ 
3: if  $a \geq b$  then
4:   return  $a$ 
5: else
6:   return  $b$ 
7: end if
  
```



This is the simplest possible flowchart pattern: a single decision diamond with two branches that both rejoin before **Stop** – the graphical form of an **if-else** statement, with no looping involved.

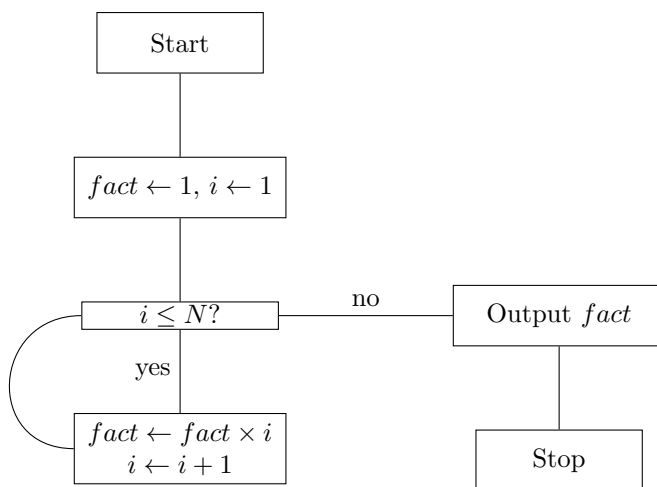
[Factorial of N – A Loop with an Accumulator]

Algorithm 4 Factorial

```

1: Input: a non-negative integer  $N$ 
2: Output:  $N! = 1 \times 2 \times \dots \times N$ 
3:  $fact \leftarrow 1$ 
4:  $i \leftarrow 1$ 
5: while  $i \leq N$  do
6:    $fact \leftarrow fact \times i$ 
7:    $i \leftarrow i + 1$ 
8: end while
9: return  $fact$ 

```



Trace by hand for $N = 4$: $(fact, i) = (1, 1) \rightarrow (1, 2) \rightarrow (2, 3) \rightarrow (6, 4) \rightarrow (24, 5)$, at which point $i \leq N$ fails ($5 \leq 4$ is false) and the loop outputs $fact = 24 = 4!$. Notice this flowchart has *exactly* the same shape as the summation flowchart in Example 1.2 – only the process-box contents differ – reinforcing that “accumulate a running value over a counted loop” is a single reusable pattern.

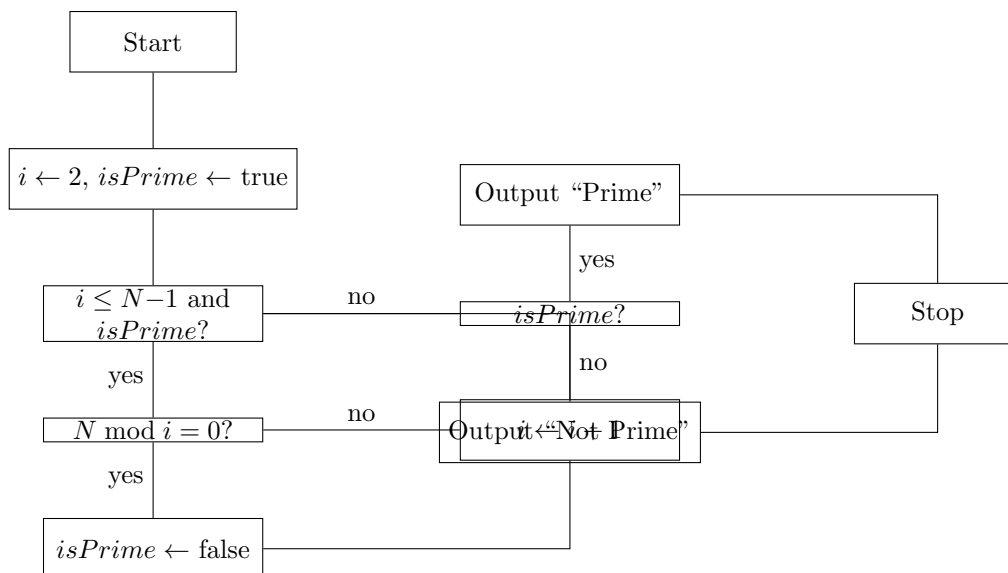
[Prime Number Check – Loop with an Early-Exit Decision]

Algorithm 5 IsPrime

```

1: Input: an integer  $N \geq 2$ 
2: Output: “Prime” or “Not Prime”
3:  $i \leftarrow 2$ 
4:  $isPrime \leftarrow \text{true}$ 
5: while  $i \leq N - 1$  and  $isPrime = \text{true}$  do
6:   if  $N \bmod i = 0$  then
7:      $isPrime \leftarrow \text{false}$ 
8:   end if
9:    $i \leftarrow i + 1$ 
10: end while
11: if  $isPrime = \text{true}$  then
12:   return “Prime”
13: else
14:   return “Not Prime”
15: end if

```



This example is more elaborate than the previous two: it has *two nested decisions* (the loop-continuation test combining two conditions with **and**, and an inner divisibility test), illustrating how flowcharts scale to represent nested control structures – the graphical counterpart of a **while** loop containing an **if** statement. Trace by hand for $N = 9$: $i = 2$ ($9 \bmod 2 = 1$, no), $i = 3$ ($9 \bmod 3 = 0$, so $isPrime \leftarrow \text{false}$, loop exits since $isPrime$ is now false), giving “Not Prime” – correctly, since $9 = 3 \times 3$.

2.1.4 Memory, Variables, Values, Instructions, and Programs

- **Memory** is a large array of addressable storage cells (bytes), each capable of holding a value.
- A **variable** is a named reference to one or more memory cells; the compiler/interpreter maps the name to an actual address.
- A **value** is the actual data (a bit pattern with a type interpretation) stored at a memory location at a given point in time.
- An **instruction** is a single operation the processor can execute (e.g. add two numbers, load a value from memory, branch to another instruction).
- A **program** is an ordered sequence of instructions, stored in memory alongside the data it operates on (the von Neumann stored-program principle).

The processor repeatedly performs a *fetch-decode-execute cycle*: fetch the next instruction from memory, decode what operation it specifies, execute that operation (possibly reading/writing memory), and advance to the next instruction. Every high-level construct a student writes – a loop, a conditional, a function call – ultimately compiles down to sequences of such primitive instructions.

Assignments

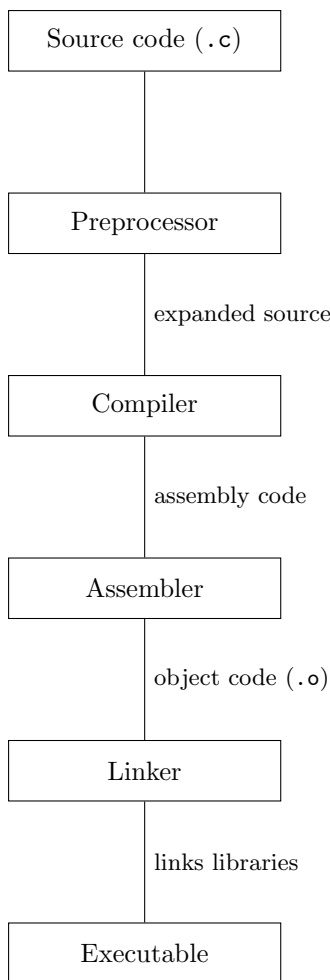
1. List, in order, the five stages of the Problem $\rightarrow$ Algorithm $\rightarrow$ Program $\rightarrow$ Execution $\rightarrow$ Result pipeline, and give a one-sentence description of what happens at each stage.
2. Write pseudocode (in the style of Algorithm 1) for finding the maximum of three input numbers a , b , c .
3. Draw a flowchart for an algorithm that determines whether a given integer n is even or odd.
4. Trace the pseudocode in Example 1.1 by hand for $N = 5$, listing the values of sum and i after each loop iteration, and confirm the final output is 15.

5. Explain, using the fetch-decode-execute cycle, what it means for a program to be “stored” in the same memory as its data (the von Neumann architecture).

2.2 Introduction to C

2.2.1 The Language of C: Phases of Developing a Running Program

C is a compiled language: source code written by a programmer must pass through several distinct phases before it becomes an executable program the operating system can run.



- **Preprocessing:** handles directives beginning with `#`, such as `#include` (textually inserts header file contents) and `#define` (macro substitution), producing expanded source text.
- **Compilation:** translates the preprocessed C code into assembly language for the target processor, checking syntax and (static) types along the way.
- **Assembly:** converts assembly code into machine-readable *object code* (`.o/.obj` files).
- **Linking:** combines one or more object files with required libraries (e.g. the C standard library, for functions like `printf`) to produce a single executable.
- **Loading and Execution:** the operating system loads the executable into memory and transfers control to it, beginning at the `main` function.

[Typical Command-Line Workflow]

```

1 $ gcc -c hello.c -o hello.o      # compile + assemble to object code
2 $ gcc hello.o -o hello           # link into an executable
3 $ ./hello                        # execute

```

In practice `gcc hello.c -o hello` performs all four phases (preprocess, compile, assemble, link) in a single command; the phases are shown separately here purely for pedagogical clarity.

2.2.2 Data Concepts in C: Constants, Variables, Expressions, Operators, and Precedence

Constants

A *constant* is a fixed value that does not change during program execution, written directly in the source code (a *literal*) or declared immutable.

```

1 42          // integer constant
2 3.14        // floating-point constant
3 'A'         // character constant
4 "Hello"     // string literal (constant array of characters)
5 const int MAX = 100; // named, immutable constant

```

Variables

As introduced in Section 2.1, a variable is a named, typed storage location. In C, every variable must be *declared* with a type before use:

```

1 int count;          // declaration only
2 int total = 0;      // declaration with initialization
3 float price = 19.99;
4 char grade = 'A';

```

Expressions

An *expression* is any combination of variables, constants, and operators that evaluates to a value.

```

1 x + y * 2          // arithmetic expression
2 (a > b) && (c < d) // logical expression, evaluates to 0 or 1
3 count++           // expression with a side effect

```

Operators and Precedence

C provides a rich set of operators; when several appear in one expression, *precedence* determines which is applied first, and *associativity* determines the order among operators of equal precedence.

Precedence	Operators	Category	Associativity
Highest	() [] -> .	Postfix	Left to right
	++ -- ! ~ (type) * &	Unary	Right to left
	* / %	Multiplicative	Left to right
	+ -	Additive	Left to right
	< <= > >=	Relational	Left to right
	== !=	Equality	Left to right
	&&	Logical AND	Left to right
		Logical OR	Left to right
	?:	Conditional (ternary)	Right to left
	= += -= *= /=	Assignment	Right to left
Lowest			

[Precedence in Action] Consider `int result = 2 + 3 * 4 > 10 && 5 != 5;`

1. `*` binds tighter than `+`: $3 \times 4 = 12$, giving `2 + 12 > 10 && 5 != 5`.
2. `+` evaluates next: $2 + 12 = 14$, giving `14 > 10 && 5 != 5`.
3. Relational/equality operators evaluate next: `14 > 10` is 1 (true), `5 != 5` is 0 (false).
4. `&&` evaluates last: `1 && 0` is 0.

So `result` is assigned 0. This kind of trace is worth doing on the board line by line so students see precedence resolving one operator at a time, not all at once.

2.2.3 Statements

Declarations

A *declaration* introduces a name (variable, function, etc.) and its type to the compiler, e.g. `int x;` or `double area;`. C requires declarations before use (traditionally at the start of a block, though modern C permits mixing declarations and code).

Input-Output Statements

The C standard library provides `scanf` (formatted input) and `printf` (formatted output).

```

1 int age;
2 printf("Enter your age: ");
3 scanf("%d", &age);           // & = address-of operator
4 printf("You are %d years old.\n", age);

```

Note the `&age` in `scanf`: the function needs the *address* of the variable (an lvalue's location) so it can write into it directly, connecting back to the lvalue/rvalue distinction introduced earlier.

Compound Statements

A *compound statement* (or *block*) groups zero or more statements inside braces `{ }`, treating them as a single statement – essential for writing multi-statement loop bodies and conditional branches.

```

1 {
2     int temp = a;
3     a = b;
4     b = temp;
5 }

```

Selection Statements: Conditions, Logical Operators, Precedence

A *condition* is any expression evaluating to a Boolean-like value (in C, 0 is false, any nonzero value is true). Logical operators `&&` (AND), `||` (OR), and `!` (NOT) combine conditions, with `!` binding tighter than `&&`, which binds tighter than `||` (see the precedence table above).

[Short-Circuit Evaluation] In `(x != 0) && (y / x > 1)`, if `x == 0` the left operand of `&&` is false, and C guarantees the right operand is *not* evaluated – avoiding a division by zero. This behaviour, called *short-circuit evaluation*, applies to both `&&` and `||` and is frequently exploited to write safe guard conditions.

Repetitive Statements

C provides three looping constructs, covered in depth in Section 2.3: `while`, `do-while`, and `for`.

Assignments

1. List, in order, the five phases a C source file passes through to become a running executable, and state what each phase produces as output.
2. Evaluate, by hand, the expression $5 + 6 / 2 - 1 * 3$ following C's operator precedence, showing each step.
3. Evaluate $(4 > 2) || (10 / 0 == 1)$ and explain, using short-circuit evaluation, why this does *not* cause a division-by-zero error.
4. Write the declaration and initialization statements for: an integer `count` set to 0, a float `pi` set to 3.14159, and a character `initial` set to 'S'.
5. Given `int a = 7, b = 2;`, compute the value of `a % b`, `a / b`, and `(a > b) ? a : b` by hand.

2.3 Conditionals and Loops

2.3.1 Conditional Statements

if, if-else, and Nested Conditionals

```

1  if (score >= 90) {
2      grade = 'A';
3  } else if (score >= 75) {
4      grade = 'B';
5  } else if (score >= 60) {
6      grade = 'C';
7  } else {
8      grade = 'F';
9  }
```

A *nested conditional* is an `if` (or `if-else`) statement placed inside the body of another; the `else if` chain above is itself syntactic sugar for nested `if-else` statements.

switch-case

When testing one variable against many discrete constant values, `switch` is often clearer than a long `if-else` if chain.

```

1  switch (day) {
2      case 1: printf("Monday");    break;
3      case 2: printf("Tuesday");   break;
4      case 3: printf("Wednesday"); break;
5      default: printf("Other day");
6  }
```

The `break` statement is essential: without it, execution “falls through” into the next case, which is occasionally used deliberately but is a common source of bugs when omitted accidentally.

2.3.2 Loops: for, while, do-while (“repeat”)

Construct	Test performed	Typical use
<code>while</code>	before each iteration	unknown number of iterations
<code>do-while</code>	after each iteration (runs ≥ 1 time)	at least one guaranteed execution
<code>for</code>	before each iteration, with init/update built in	known/counted number of iterations

```

1 // while: test before body
2 int i = 0;
3 while (i < 5) { printf("%d ", i); i++; }
4
5 // do-while: body runs at least once
6 int j = 0;
7 do { printf("%d ", j); j++; } while (j < 5);
8
9 // for: initialization, condition, update in one line
10 for (int k = 0; k < 5; k++) { printf("%d ", k); }

```

All three loops above print 0 1 2 3 4; the difference lies in *when* the condition is tested and where the loop-control variable's update is written, not in the final output for this particular example.

[When do-while Matters]

```

1 int choice;
2 do {
3     printf("Enter 1-5: ");
4     scanf("%d", &choice);
5 } while (choice < 1 || choice > 5);

```

Here **do-while** is the natural choice: the prompt must execute *at least once* before there is any value of *choice* to test, which a **while** loop (which tests *before* the first iteration) cannot express as directly.

2.3.3 Loop Invariants, Preconditions, and Postconditions

This subsection follows the formal reasoning style developed in Gries' *The Science of Programming* and in Dahl, Dijkstra, and Hoare's *Structured Programming*, where loops are not just executed and tested empirically but *proven correct* using assertions.

[Precondition and Postcondition] A *precondition* P is an assertion that is assumed to hold *before* a statement (or program) executes. A *postcondition* Q is an assertion guaranteed to hold *after* the statement executes, provided P held beforehand. We write this as a Hoare triple: $\{P\} S \{Q\}$.

[Loop Invariant] A *loop invariant* is an assertion I that is true (i) immediately before the loop begins, (ii) after every iteration of the loop body, and therefore (iii) when the loop terminates (at which point it combines with the negated loop condition to establish the postcondition). Loop invariants are the primary tool for *proving* that a loop computes the correct result, rather than merely observing correct output on test cases.

[Loop Invariant for Summation] Reconsider the summation loop from Section 2.1:

```

1 int sum = 0, i = 1;
2 while (i <= N) {
3     sum = sum + i;
4     i = i + 1;
5 }

```

Precondition: $N \geq 0$.

Loop invariant I : at the top of each iteration, $sum = 1 + 2 + \dots + (i - 1) = \frac{(i-1)i}{2}$, and $1 \leq i \leq N + 1$.

- *Initialization:* before the first iteration, $i = 1$ and $sum = 0 = \frac{0 \cdot 1}{2}$, so I holds.
- *Maintenance:* if I holds with some i , after executing the body, sum becomes $\frac{(i-1)i}{2} + i = \frac{i(i+1)}{2}$ and i becomes $i + 1$, so I again holds with the new i .
- *Termination:* the loop exits when $i = N + 1$, at which point I gives $sum = \frac{N(N+1)}{2}$, exactly the closed-form sum of the first N integers – our desired **postcondition**.

This invariant-based argument *proves* the loop correct for all valid N , rather than checking a few sample values – the central methodological point of the Gries/Dijkstra formal style.

Assignments

1. Rewrite the **if-else if** chain that assigns a letter grade (Section 3.1.1) as a fully nested sequence of **if-else** statements (no **else if**), and confirm it produces identical behaviour for **score = 82**.
2. Convert the **switch** statement for days of the week into an equivalent **if-else if** chain, and explain what would change in the output if the **break** after **case 2** were omitted, for **day = 2**.
3. Trace the **do-while** menu-prompt example by hand assuming the user enters 8, then 0, then 3; list what is printed and when the loop terminates.
4. Write a **for** loop that computes the product $1 \times 2 \times 3 \times \dots \times 6$ (i.e. $6!$), and state a loop invariant analogous to the summation example that would let you prove the loop computes $6! = 720$ correctly.
5. For the loop `for (int i = 1; i <= N; i++) sum += i*i;`, state a suitable loop invariant expressing *sum* in terms of *i*, and verify it numerically for $N = 4$ by tracing $i = 1, 2, 3, 4$ and computing *sum* at each step (expected final value: $1 + 4 + 9 + 16 = 30$).

2.4 Data Types, Size and Values

This section revisits and extends the number-representation material from Chapter 1, specializing it to C's concrete type system.

2.4.1 char, Signed, and Unsigned Types

C's basic integer types can each be qualified **signed** or **unsigned** (**signed** is the default for **int** and is usually, but not always, the default for **char**, which is why **signed char** / **unsigned char** should be stated explicitly when the sign matters).

Type	Typical size	Signed range	Unsigned range
char	1 byte	$[-128, 127]$	$[0, 255]$
short	2 bytes	$[-32768, 32767]$	$[0, 65535]$
int	4 bytes	$[-2^{31}, 2^{31} - 1]$	$[0, 2^{32} - 1]$
long long	8 bytes	$[-2^{63}, 2^{63} - 1]$	$[0, 2^{64} - 1]$

```

1 signed char sc = -100;      // valid: within [-128, 127]
2 unsigned char uc = 250;    // valid: within [0, 255]
3 unsigned int ui = -1;      // legal syntax, but wraps to UINT_MAX (4294967295)!
```

[Signed/Unsigned Mismatch Pitfall]

```

1 unsigned int u = 3;
2 int i = -5;
3 if (i < u)
4     printf("i is smaller\n");
5 else
6     printf("u is smaller\n");    // this branch actually executes!
```

In a mixed comparison, C *implicitly converts* the signed **int** to **unsigned int** before comparing. Since -5 as an unsigned 32-bit value becomes a very large positive number (4,294,967,291), the condition $i < u$ is *false*, printing “u is smaller” – a classic real-world bug caused by silent signed/unsigned conversion rules.

2.4.2 Number Systems and Representations

As covered in detail in Chapter 1 (binary, octal, decimal, hexadecimal, and two's complement), C allows integer literals to be written directly in multiple bases:

```

1 int dec = 42;           // decimal (default)
2 int oct = 052;         // octal: leading 0 -> value 42
3 int hex = 0x2A;        // hexadecimal: leading 0x -> value 42
4 int bin = 0b101010;    // binary: leading 0b (C23 / common extension) -> value 42

```

All four literals above represent the same value, 42; only the notation used to write it in source code differs.

2.4.3 Constants and Overflow, Revisited in C

```

1 #include <limits.h>
2 printf("%d\n", INT_MAX);           // largest representable int, e.g. 2147483647
3 printf("%d\n", INT_MAX + 1);       // undefined behaviour: typically wraps to INT_MIN

```

The header `<limits.h>` provides named constants (`INT_MAX`, `INT_MIN`, `UINT_MAX`, `CHAR_MAX`, etc.) so that overflow-sensitive code can check bounds explicitly rather than relying on the wraparound behaviour discussed in Chapter 1, which is well-defined for unsigned types but *undefined* for signed types in C.

Assignments

1. Convert the octal literal `052` and the hexadecimal literal `0x2A` to decimal by hand, and confirm both equal 42.
2. Given `unsigned char uc = 200;`, compute the value of `uc + 100` using modulo-256 arithmetic (as in Chapter 1), and state the resulting `unsigned char` value.
3. Explain what value is printed by `unsigned int ui = -1; printf("%u", ui);` on a system with 32-bit `unsigned int`, and show the arithmetic that produces it.
4. In the signed/unsigned mismatch example, replace `i = -5` with `i = -1` and `u = 3`. Determine, showing your reasoning, which branch of the `if` statement executes.
5. If `INT_MAX = 2147483647` on a given system, what is the result of `INT_MAX + 1` assuming two's-complement wraparound behaviour (even though this is undefined behaviour in standard C)?

2.5 Arrays, Strings, and Multidimensional Arrays

2.5.1 One-Dimensional Arrays

[Array] An *array* is a fixed-size, contiguous block of memory holding multiple elements of the *same* type, accessed by an integer *index* starting at 0.

```

1 int scores[5] = {90, 85, 77, 92, 60};
2 printf("%d\n", scores[0]);    // 90 (first element)
3 printf("%d\n", scores[4]);    // 60 (last element, index size-1)
4 scores[2] = 80;               // modify the third element

```

Because array elements are stored contiguously, the address of element `scores[i]` is computed as `base_address + i × sizeof(element type)` – a direct application of the memory-addressing ideas from Section 2.1.

[Array Bounds – A Common Bug]

```

1 int arr[5] = {1, 2, 3, 4, 5};
2 printf("%d\n", arr[5]);       // out of bounds! valid indices are 0..4

```


C performs *no* automatic bounds checking; reading or writing `arr[5]` accesses whatever memory happens to follow the array, which is undefined behaviour and a leading cause of security vulnerabilities (buffer overflows) in C programs.

2.5.2 Strings as Character Arrays

C has no dedicated string type; a *string* is simply a `char` array terminated by a special *null character* `'\0'` (numeric value 0) marking its end.

```
1 char name[6] = "Alice";    // stored as {'A','l','i','c','e','\0'} -- 6 bytes
2 printf("%s\n", name);      // %s prints characters until '\0' is found
3 printf("%d\n", strlen(name)); // 5 -- length excludes the null terminator
```

[Why the Null Terminator Matters] Given `char buf[4] = {'H','i','\0','X'};`, `printf("%s", buf)` prints `Hi` and stops at index 2 – the character `'X'` at index 3 is never reached, because string-handling functions rely entirely on locating `'\0'`, not on knowing the array's declared size.

2.5.3 Multidimensional Arrays and Matrices

A two-dimensional array models a matrix: `int m[ROWS][COLS]`. In memory, C stores 2-D arrays in *row-major order* – all of row 0, then all of row 1, and so on.

```
1 int m[2][3] = {
2     {1, 2, 3},
3     {4, 5, 6}
4 };
5 printf("%d\n", m[1][2]);    // 6 (row index 1, column index 2)
```

[Row-Major Address Computation] For `int m[2][3]`, the address of `m[i][j]` is

$$\text{base_address} + (i \times 3 + j) \times \text{sizeof}(\text{int}).$$

For `m[1][2]`: $\text{offset} = (1 \times 3 + 2) = 5$ elements from the base, i.e. the 6th `int` stored, consistent with row-major (row-by-row) layout: $\underbrace{1, 2, 3}_{\text{row 0}}, \underbrace{4, 5, 6}_{\text{row 1}}$, where 6 sits at linear position 5 (0-indexed).

[Matrix Addition]

```
1 #define N 2
2 int A[N][N] = {{1,2},{3,4}};
3 int B[N][N] = {{5,6},{7,8}};
4 int C[N][N];
5
6 for (int i = 0; i < N; i++)
7     for (int j = 0; j < N; j++)
8         C[i][j] = A[i][j] + B[i][j];
9 // C = {{6,8},{10,12}}
```

This nested-loop pattern – outer loop over rows, inner loop over columns – is the standard idiom for processing every element of a matrix, and directly reuses the `for`-loop and loop-invariant reasoning developed in Section 2.3, now with a two-dimensional index.

Assignments

1. Given `int arr[6] = {4, 8, 15, 16, 23, 42};`, write the value and index of the element equal to 16, and compute the byte offset of `arr[4]` from the base address assuming `sizeof(int) = 4`.
2. Write C code to compute the sum of all elements in an integer array of size $n = 7$, and trace it by hand for $\{2, 4, 6, 8, 10, 12, 14\}$, confirming the final sum is 56.

3. Given `char word[6] = "Hello";`, state the value stored at `word[5]` and explain its role.
4. For `int m[3][3]`, derive the formula for the linear (row-major) offset of `m[i][j]`, and use it to compute the offset of `m[2][1]`.
5. Write C code to transpose a 2×3 matrix `int A[2][3]` into a 3×2 matrix `int T[3][2]`, and trace it by hand for `A = {{1,2,3},{4,5,6}}`, writing out the resulting `T`.

References and Notes on Coverage

- **R. G. Dromey**, *How to Solve it by Computer* – primary source for the Problem $\rightarrow$ Algorithm $\rightarrow$ Program pipeline, pseudocode, and flowchart-based problem decomposition in Section 2.1.
- **D. Gries**, *The Science of Programming* – primary source for the formal treatment of preconditions, postconditions, and loop invariants in Section 2.3.
- **O. J. Dahl, E. W. Dijkstra, C. A. R. Hoare**, *Structured Programming* – underlies the structured control-flow philosophy (avoiding unstructured jumps in favour of sequencing, selection, and iteration) reflected throughout Sections 2.2–2.3.
- **E. W. Dijkstra**, *A Short Introduction to the Art of Programming* – secondary reference for the guarded-command style of reasoning about conditionals and loops.

Coverage note: the C-specific material in Sections 2.2–2.5 (concrete syntax for declarations, `scanf/printf`, `switch-case`, and array/string mechanics) is *not* contained in the six reference books listed for this course – they focus on programming methodology and (for Goodrich/Standish) Java, not C syntax. For syntax-level reference and further practice problems, a standard C textbook such as Kernighan & Ritchie’s *The C Programming Language* is recommended as a supplementary text.

Chapter 3

Functions, Recursions, Sorting and Search

1. The prototype declaration, Function definition
2. Function call: Passing arguments to a function, by value, by reference. Scope of variable names. Recursive function calls, Tail recursion. Analysing recursion, Tree of recursion, linear recursion
3. Sorting problem: Selection Sort, Insertion Sort, Comparison between sorting algorithms. Sorting in multidimensional arrays. Sorting in strings.
4. Search problem: Linear search and binary search. Comparison between search procedures. Recursive and Iterative formulations.

3.1 Functions: Prototypes and Definitions

3.1.1 Why Functions?

As programs grow, repeating the same block of code in multiple places becomes error-prone and hard to maintain. A *function* packages a computation under a name, so it can be invoked (“called”) from many places with different inputs. This idea of decomposing a large problem into smaller, independently understandable pieces is central to structured programming, as developed in Dahl, Dijkstra, and Hoare’s *Structured Programming* and reflected in Dromey’s problem-decomposition approach in *How to Solve it by Computer*.

3.1.2 Function Prototype Declaration

[Function Prototype] A *function prototype* (or *declaration*) specifies a function’s name, return type, and parameter types, *without* providing its body. It tells the compiler “a function with this signature exists,” allowing calls to be type-checked before the function’s full definition is seen.

```
1 // Prototypes (typically placed near the top of a file, or in a header)
2 int add(int a, int b);
3 double average(double x, double y, double z);
4 void printLine(int length);
```

3.1.3 Function Definition

[Function Definition] A *function definition* provides the full implementation: the signature (matching its prototype) together with a body of statements, including a **return** statement if the function produces a value.

[Prototype and Definition Together]

```

1 #include <stdio.h>
2
3 int add(int a, int b);    // prototype
4
5 int main(void) {
6     int result = add(3, 4);    // call: uses the prototype to type-check
7     printf("%d\n", result);    // prints 7
8     return 0;
9 }
10
11 int add(int a, int b) {    // definition: appears after main, but that's fine
12     return a + b;
13 }

```

Because the prototype appears before `main`, the compiler already knows `add` takes two `ints` and returns an `int` when it type-checks the call inside `main`, even though the *definition* (with the actual body) appears later in the file. Without the prototype, calling a function before its definition is a compile error (or, in older C standards, produces an incorrect implicit-`int` assumption).

[A Function with No Return Value]

```

1 void printStars(int n) {    // 'void' -- returns nothing
2     for (int i = 0; i < n; i++) {
3         printf("*");
4     }
5     printf("\n");
6 }
7
8 int main(void) {
9     printStars(5);    // prints: *****
10    return 0;
11 }

```

Assignments

1. Write a function prototype and definition for `int square(int x)` that returns x^2 , and a `main` that prints `square(7)`. State the expected output.
2. Explain, in your own words, why a prototype lets a function be *called* before it is *defined* later in the same file.
3. Write a `void` function `printTable(int n)` that prints the multiplication table of n from $1 \times n$ to $10 \times n$. Trace its output by hand for $n = 3$ (first three lines only).

3.2 Function Calls: Arguments, Scope, and Recursion

3.2.1 Passing Arguments by Value

[Pass by Value] In *pass by value*, the function receives a *copy* of the argument's value. Any modification to the parameter inside the function has no effect on the caller's original variable.

[Pass by Value – No Effect on the Caller]

```

1 void increment(int x) {
2     x = x + 1;    // modifies the local copy only
3     printf("Inside: x = %d\n", x);
4 }
5

```

```

6 int main(void) {
7     int a = 5;
8     increment(a);
9     printf("Outside: a = %d\n", a);
10    return 0;
11 }

```

Output:

```

1 Inside: x = 6
2 Outside: a = 5

```

The variable `a` in `main` is untouched, because `x` inside `increment` is a separate copy occupying its own memory location.

3.2.2 Passing Arguments by Reference (via Pointers)

C has no native “pass by reference” keyword (unlike C++), but the same effect is achieved by passing a *pointer* – the address of a variable – so the function can modify the caller’s original data through that address.

[Simulated Pass by Reference – The Classic Swap]

```

1 void swap(int *px, int *py) {
2     int temp = *px;    // *px dereferences: "the value pointed to by px"
3     *px = *py;
4     *py = temp;
5 }
6
7 int main(void) {
8     int a = 5, b = 9;
9     swap(&a, &b);      // &a: "address of a"
10    printf("a = %d, b = %d\n", a, b);    // a = 9, b = 5
11    return 0;
12 }

```

Here `swap` receives the *addresses* of `a` and `b`, not copies of their values, so modifications made through `*px` and `*py` are visible in `main` after the call returns. This connects directly to the lvalue/address-of operator (`&`) introduced in Chapter 1 and the `scanf(..., &var)` pattern from Chapter 2.

	Pass by value	Pass by reference (pointer)
What is copied	the value	the address
Can modify caller’s variable?	No	Yes
Typical use	simple inputs (e.g. <code>int</code> , <code>double</code>)	outputs, large structures, arrays

Note: arrays are always effectively passed “by reference” in C, because an array argument automatically decays to a pointer to its first element – a subtlety worth flagging explicitly, since it looks like ordinary pass-by-value syntax but behaves differently from scalar types.

3.2.3 Scope of Variable Names

[Scope] The *scope* of a variable is the region of program text within which its name is visible and can be referenced.

- **Local (block) scope:** a variable declared inside a function or block `{ }` is visible only within that block, and ceases to exist once the block exits.
- **Global (file) scope:** a variable declared outside any function is visible to all functions in the file (and other files, if declared `extern`) from its point of declaration onward.

[Local Variables and Shadowing]

```

1 int x = 100;                // global x
2
3 void demo(void) {
4     int x = 5;              // local x "shadows" the global x
5     printf("Local x = %d\n", x);    // 5
6     {
7         int x = 1;          // an even more local x, in an inner block
8         printf("Inner x = %d\n", x);    // 1
9     }
10    printf("Local x again = %d\n", x); // 5 (inner block's x is gone)
11 }
12
13 int main(void) {
14     demo();
15     printf("Global x = %d\n", x);    // 100 -- untouched by demo()
16     return 0;
17 }

```

Each block introduces its own scope; an inner declaration temporarily *shadows* (hides) an outer variable of the same name without altering it, and the shadow disappears once its block ends.

3.2.4 Recursive Function Calls

[Recursion] A function is *recursive* if it calls itself, either directly or indirectly (through another function). Every correct recursive function requires: (i) one or more *base cases* that terminate the recursion directly, without further recursive calls, and (ii) a *recursive case* that reduces the problem toward a base case.

[Factorial – The Canonical Recursive Function]

```

1 int factorial(int n) {
2     if (n == 0) {           // base case
3         return 1;
4     }
5     return n * factorial(n - 1); // recursive case
6 }

```

Calling `factorial(4)` unwinds as: $4 \times \text{factorial}(3) = 4 \times (3 \times \text{factorial}(2)) = \dots = 4 \times 3 \times 2 \times 1 \times \text{factorial}(0) = 4 \times 3 \times 2 \times 1 \times 1 = 24$.

Tail Recursion

[Tail Recursion] A recursive call is a *tail call* if it is the *last* operation performed in the function – nothing (like a pending multiplication) remains to be done after the recursive call returns. A function consisting entirely of such calls is *tail-recursive*.

[Non-Tail vs. Tail-Recursive Factorial]

```

1 // Non-tail-recursive: multiplication happens AFTER the recursive call returns
2 int factorial(int n) {
3     if (n == 0) return 1;
4     return n * factorial(n - 1);    // pending multiplication -- not a tail call
5 }
6
7 // Tail-recursive: the recursive call is the very last action, result
8 // is threaded through an accumulator parameter
9 int factorialTail(int n, int acc) {
10    if (n == 0) return acc;
11    return factorialTail(n - 1, n * acc);    // nothing left to do after this call
12 }

```

```
13 // initial call: factorialTail(4, 1)
```

In the tail-recursive version, each call’s result *is* the final result – there is no “pending” work waiting for the recursive call to return, which in principle allows a compiler to reuse the current function’s stack frame for the next call (*tail-call optimization*), turning the recursion into an iteration internally. Standard C compilers are *not required* to perform this optimization, so deep tail recursion can still overflow the call stack in C, unlike in languages that guarantee tail-call elimination.

Analysing Recursion: Linear Recursion and the Tree of Recursion

[Linear Recursion] A function exhibits *linear recursion* if each call makes *at most one* recursive call. The sequence of calls then forms a single chain (a “linear” recursion depth), as in `factorial` above.

[Tree Recursion] A function exhibits *tree recursion* if a call can make *more than one* recursive call. The set of all calls forms a branching call *tree* rather than a simple chain.

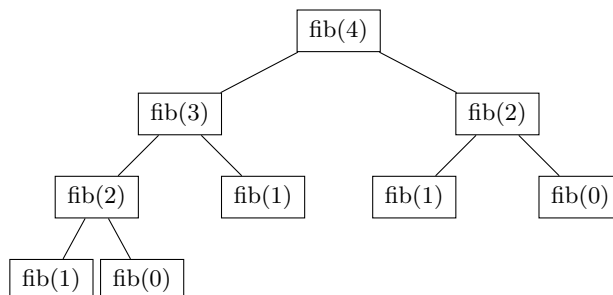
[Linear Recursion – Call Chain for `factorial(4)`]

$$factorial(4) \rightarrow factorial(3) \rightarrow factorial(2) \rightarrow factorial(1) \rightarrow factorial(0)$$

Each level makes exactly one recursive call, so the recursion depth is 4, and the concrete complexity (number of calls) is $\Theta(n)$ – *linear* in n , matching the name.

[Tree Recursion – Naive Fibonacci]

```
1 int fib(int n) {
2     if (n <= 1) return n;           // base cases: fib(0)=0, fib(1)=1
3     return fib(n - 1) + fib(n - 2); // TWO recursive calls
4 }
```



The call tree for `fib(4)` branches at every internal node, and notice `fib(2)` and `fib(1)` are each recomputed *multiple times independently* – a hallmark inefficiency of naive tree recursion. Counting nodes, this tree has 9 calls for $n = 4$; in general the number of calls grows as $O(2^n)$ (exponential), since the recurrence $T(n) = T(n - 1) + T(n - 2) + O(1)$ roughly doubles the work at each level – in sharp contrast to the $\Theta(n)$ linear recursion of `factorial`. This is a natural place to connect back to the big-O analysis of Chapter 1: the *shape* of the recursion tree (chain vs. branching) directly determines the asymptotic complexity class.

Reasoning About Recursive Correctness

Following the style of formal reasoning in Gries’ *The Science of Programming* (recursion induction) and Dijkstra’s writings, a recursive function can be proven correct much like mathematical induction: show the base case is correct, and show that *if* the recursive call(s) return correct results for smaller inputs, *then* the current call combines them correctly. This mirrors the loop-invariant style of proof used for iterative loops in Chapter 2, applied instead to the “smaller subproblem” structure of recursion.

Assignments

1. Trace the pass-by-value `increment` example by hand for `a = 20`, stating the values printed inside and outside the function.

2. Write a void function `tripleInPlace(int *p)` that triples the value pointed to by `p`, and a `main` that calls it on a variable `v = 7`, printing the result (21).
3. In the shadowing example, if the innermost block instead read `x = x + 1`; (referring to the local `x = 5`), what would `Inner x` print, and what would `Local x` again print afterward?
4. Trace `factorial(5)` by hand, writing out the full chain of multiplications, and confirm the result is 120.
5. Rewrite `factorial` as a tail-recursive function `factorialTail(n, acc)` (as shown in the notes) and trace the sequence of calls for `factorialTail(4, 1)`, listing (n, acc) at each call.
6. Draw the recursion tree for `fib(5)`, count the total number of calls (including base cases), and compare it to the count for `fib(4)` (which is 9).
7. Is the function `int sumDigits(int n) { if (n == 0) return 0; return n%10 + sumDigits(n/10); }` linearly recursive or tree recursive? Justify your answer, and trace it for $n = 407$.

3.3 Sorting Algorithms

3.3.1 The Sorting Problem

[Sorting] Given a sequence of n comparable elements, *sorting* rearranges them into non-decreasing (or non-increasing) order.

Sorting is one of the most thoroughly studied problems in computer science and a natural vehicle for applying the complexity-analysis tools from Chapter 1; the treatment below follows the algorithmic style of Goodrich's *Data Structures and Algorithms* and Standish's *Data Structures in Java*, adapted here to C.

3.3.2 Selection Sort

[Selection Sort] *Selection sort* repeatedly finds the minimum element in the unsorted portion of the array and swaps it into its correct sorted position, growing the sorted prefix by one element per pass.

```

1 void selectionSort(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         int minIdx = i;
4         for (int j = i + 1; j < n; j++) {
5             if (arr[j] < arr[minIdx]) {
6                 minIdx = j;
7             }
8         }
9         // swap arr[i] and arr[minIdx]
10        int temp = arr[i];
11        arr[i] = arr[minIdx];
12        arr[minIdx] = temp;
13    }
14 }
```

[Selection Sort – Hand Trace] Sort [29, 10, 14, 37, 13]:

Pass	Minimum found	Array after swap
$i = 0$	10 (index 1)	[10, 29, 14, 37, 13]
$i = 1$	13 (index 4)	[10, 13, 14, 37, 29]
$i = 2$	14 (already in place)	[10, 13, 14, 37, 29]
$i = 3$	29 (index 4)	[10, 13, 14, 29, 37]

Final sorted array: [10, 13, 14, 29, 37], achieved in $n - 1 = 4$ passes.

Complexity: the inner loop always scans the remaining unsorted elements regardless of their order, giving $\sum_{i=0}^{n-2} (n - 1 - i) = \frac{n(n-1)}{2}$ comparisons in *every* case – best, average, and worst are all $\Theta(n^2)$. Selection sort performs at most $n - 1$ swaps, making it attractive when swaps are expensive (e.g. large records) even though comparisons are not reduced.

3.3.3 Insertion Sort

[Insertion Sort] *Insertion sort* builds the sorted portion one element at a time, taking the next unsorted element and shifting it leftward past all larger elements in the sorted prefix until it reaches its correct position.

```

1 void insertionSort(int arr[], int n) {
2     for (int i = 1; i < n; i++) {
3         int key = arr[i];
4         int j = i - 1;
5         while (j >= 0 && arr[j] > key) {
6             arr[j + 1] = arr[j];    // shift larger element right
7             j = j - 1;
8         }
9         arr[j + 1] = key;           // insert key in its correct slot
10    }
11 }
```

[Insertion Sort – Hand Trace] Sort [29, 10, 14, 37, 13]:

Pass	key	Array after insertion
$i = 1$	10	[10, 29, 14, 37, 13]
$i = 2$	14	[10, 14, 29, 37, 13]
$i = 3$	37	[10, 14, 29, 37, 13] (already in place)
$i = 4$	13	[10, 13, 14, 29, 37]

Complexity: worst case (reverse-sorted input) requires shifting every prior element for each insertion, giving $\Theta(n^2)$ comparisons/shifts. Best case (already sorted input) requires only *one* comparison per outer iteration (the **while** condition fails immediately), giving $\Theta(n)$ – insertion sort is notably efficient on nearly-sorted data, unlike selection sort, whose behaviour is identical regardless of input order.

3.3.4 Comparison Between Sorting Algorithms

Algorithm	Best	Average	Worst	Stable?
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	No (naive swap-based version)
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	Yes

[Stability] A sorting algorithm is *stable* if it preserves the relative order of elements that compare as equal. Stability matters when sorting records by one field while wanting ties broken by their original order (e.g. sorting students by grade, preserving alphabetical order among same-grade students from a prior sort).

Practical guidance: despite matching worst-case complexity, insertion sort is generally preferred over selection sort in practice for small or nearly-sorted arrays, because of its adaptive best case and stability; selection sort's only practical advantage is minimizing the number of swaps.

3.3.5 Sorting in Multidimensional Arrays

A common task is sorting the *rows* of a 2-D array (matrix) according to some key, such as the first column, rather than sorting individual numbers.

[Sorting Rows of a Matrix by First Column]

```

1 #define ROWS 4
2 #define COLS 3
3
4 void sortRowsByFirstColumn(int m[ROWS][COLS]) {
5     for (int i = 0; i < ROWS - 1; i++) {
6         int minIdx = i;
7         for (int j = i + 1; j < ROWS; j++) {
8             if (m[j][0] < m[minIdx][0]) { // compare using first column
9                 minIdx = j;
10            }
11        }
12        if (minIdx != i) {
13            for (int k = 0; k < COLS; k++) { // swap ENTIRE rows
14                int temp = m[i][k];
15                m[i][k] = m[minIdx][k];
16                m[minIdx][k] = temp;
17            }
18        }
19    }
20 }
21 // Example: {{5,1,2},{2,9,9},{8,0,0},{1,4,4}}
22 // sorts (by column 0) to: {{1,4,4},{2,9,9},{5,1,2},{8,0,0}}

```

This is exactly selection sort's structure (Section 3.2), with the scalar comparison `arr[j] < arr[minIdx]` replaced by a key-column comparison `m[j][0] < m[minIdx][0]`, and the scalar swap replaced by a loop that swaps every element of the two rows – illustrating how the same sorting *pattern* generalizes once the notion of “element” is generalized from a number to a row.

3.3.6 Sorting Strings

Two distinct tasks are both called “sorting strings”: sorting the *characters within* a single string, and sorting an *array of strings* (words) into alphabetical order.

[Sorting Characters Within a String]

```

1 #include <string.h>
2
3 void sortCharsInString(char s[]) {
4     int n = strlen(s);
5     insertionSort((int *)0, 0); // (not used directly -- adapt below)
6 }
7
8 // Direct adaptation of insertion sort to characters:
9 void sortChars(char s[]) {
10    int n = strlen(s);
11    for (int i = 1; i < n; i++) {
12        char key = s[i];
13        int j = i - 1;
14        while (j >= 0 && s[j] > key) {
15            s[j + 1] = s[j];
16            j--;
17        }
18        s[j + 1] = key;
19    }
20 }
21 // sortChars("banana") -> "aaabnn"

```

[Sorting an Array of Strings Alphabetically]

```

1  #include <string.h>
2
3  void sortWords(char *words[], int n) {
4      for (int i = 0; i < n - 1; i++) {
5          int minIdx = i;
6          for (int j = i + 1; j < n; j++) {
7              if (strcmp(words[j], words[minIdx]) < 0) { // strcmp, not <
8                  minIdx = j;
9              }
10         }
11         char *temp = words[i]; // swap POINTERS, not string contents
12         words[i] = words[minIdx];
13         words[minIdx] = temp;
14     }
15 }
16
17 int main(void) {
18     char *fruits[] = {"banana", "apple", "cherry", "date"};
19     sortWords(fruits, 4);
20     for (int i = 0; i < 4; i++) printf("%s\n", fruits[i]);
21     // Output: apple, banana, cherry, date
22     return 0;
23 }

```

Since C strings cannot be compared with `<` (that would compare pointer addresses, not textual content), `strcmp` is used instead: it returns a negative value if the first string is lexicographically smaller, zero if equal, and positive if larger – a direct analogue of the numeric comparison in selection sort, substituting `strcmp(a,b) < 0` wherever we previously wrote `a < b`.

Assignments

1. Trace selection sort by hand on [8, 3, 6, 1, 9], showing the array after each pass, and state the total number of comparisons made (using $n(n-1)/2$ for $n = 5$).
2. Trace insertion sort by hand on [8, 3, 6, 1, 9], showing the array after each outer iteration.
3. For the already-sorted array [1, 2, 3, 4, 5], count the number of comparisons insertion sort performs, and compare it to selection sort's comparison count for the same array. Which algorithm benefits from already-sorted input, and why?
4. Using the row-sorting example, sort the matrix $\{\{7,1\}, \{2,5\}, \{4,0\}\}$ by its first column by hand, showing the matrix after each pass.
5. Trace `sortChars` on the string "cab", showing the string after each outer iteration, and confirm the final result is "abc".
6. Given the word array {"pear", "kiwi", "fig"}, trace `sortWords` by hand (using lexicographic comparison), listing the array after each pass.

3.4 Searching Algorithms

3.4.1 The Search Problem

[Searching] Given a collection of n elements and a target value, *searching* determines whether the target is present and, if so, its position.

3.4.2 Linear Search

Linear search examines elements one at a time from the start of the array until the target is found or the array is exhausted. It requires no assumption about the array's order.

[Linear Search – Iterative]

```

1 int linearSearchIter(int arr[], int n, int target) {
2     for (int i = 0; i < n; i++) {
3         if (arr[i] == target) {
4             return i;        // found at index i
5         }
6     }
7     return -1;              // not found
8 }
```

[Linear Search – Recursive]

```

1 int linearSearchRec(int arr[], int n, int target) {
2     if (n <= 0) {
3         return -1;          // base case: nothing left to search
4     }
5     if (arr[0] == target) {
6         return 0;           // base case: found at front
7     }
8     int result = linearSearchRec(arr + 1, n - 1, target); // search the rest
9     return (result == -1) ? -1 : result + 1; // adjust index for the offset
10 }
```

The recursive version is *linearly recursive* (Section 3.2): each call makes at most one further call, on an array one element smaller, mirroring the iterative version's single pass but expressed as reduction to a smaller subproblem.

Complexity: worst case (target absent, or at the last position) is $\Theta(n)$; best case (target at index 0) is $\Theta(1)$.

3.4.3 Binary Search

Binary search requires the array to be *sorted*. It repeatedly compares the target to the middle element, discarding half of the remaining search space at each step.

[Binary Search – Iterative]

```

1 int binarySearchIter(int arr[], int n, int target) {
2     int lo = 0, hi = n - 1;
3     while (lo <= hi) {
4         int mid = lo + (hi - lo) / 2; // avoids overflow vs (lo+hi)/2
5         if (arr[mid] == target) {
6             return mid;
7         } else if (arr[mid] < target) {
8             lo = mid + 1;
9         } else {
10            hi = mid - 1;
11        }
12    }
13    return -1;
14 }
```

[Binary Search – Recursive]

```

1 int binarySearchRec(int arr[], int lo, int hi, int target) {
2     if (lo > hi) {
3         return -1;          // base case: search space exhausted
4     }
```

```

4   }
5   int mid = lo + (hi - lo) / 2;
6   if (arr[mid] == target) {
7       return mid;                      // base case: found
8   } else if (arr[mid] < target) {
9       return binarySearchRec(arr, mid + 1, hi, target);    // search right half
10  } else {
11      return binarySearchRec(arr, lo, mid - 1, target);    // search left half
12  }
13 }
14 // initial call: binarySearchRec(arr, 0, n - 1, target)

```

[Binary Search – Hand Trace] Search for 23 in [2, 5, 8, 12, 16, 23, 38, 45, 56, 72] ($n = 10$, indices 0–9):

Step	lo	hi	mid	arr[mid]
1	0	9	4	16 < 23, so $lo \leftarrow 5$
2	5	9	7	45 > 23, so $hi \leftarrow 6$
3	5	6	5	23 = 23, found at index 5

Only 3 comparisons were needed for $n = 10$ elements, versus up to 6 (index of 23, plus one) for linear search on the same array.

3.4.4 Comparison Between Search Procedures

	Linear Search	Binary Search
Requires sorted input?	No	Yes
Worst-case complexity	$\Theta(n)$	$\Theta(\log n)$
Best-case complexity	$\Theta(1)$	$\Theta(1)$
Extra cost	none	must sort first if unsorted, $\Theta(n \log n)$

Practical guidance: if the array is searched only *once*, linear search is often preferable, since sorting first (to enable binary search) costs $\Theta(n \log n)$ – more than a single linear scan. Binary search’s advantage compounds when the *same sorted array* is searched *many* times, amortizing the one-time sorting cost, or when the data arrives already sorted.

3.4.5 Recursive vs. Iterative Formulations

Both linear and binary search were shown above in *iterative* (loop-based) and *recursive* (self-calling) forms, computing identical results.

	Iterative	Recursive
Control mechanism	explicit loop	function calls itself
Memory overhead	$O(1)$ extra space	$O(\log n)$ stack frames (binary search) or $O(n)$ stack frames (linear search)
Readability	sometimes less direct	often mirrors the problem’s natural recursive structure

Binary search’s recursive formulation is a particularly clean illustration of *linear recursion* (Section 3.2) applied to search: each call reduces the problem to a subproblem *half* the size, giving the recurrence $T(n) = T(n/2) + O(1)$, which solves to $T(n) = \Theta(\log n)$ – the same complexity as the iterative version, since the recursive calls here are tail calls and could, in principle, be compiled into the equivalent loop.

Assignments

1. Trace `linearSearchIter` by hand on `[4, 17, 9, 23, 2]` searching for 23, stating how many comparisons are made and the returned index.
2. Trace `linearSearchRec` on the same array and target as the previous question, listing the sequence of recursive calls (with the shrinking array/length) until the base case is reached.
3. Trace `binarySearchIter` by hand searching for 12 in `[1, 4, 6, 8, 12, 15, 19, 24]` ($n = 8$), showing `lo`, `hi`, and `mid` at each step until found.
4. Repeat the previous question searching for 5 (not present), showing how `lo` and `hi` cross and the loop terminates with `-1`.
5. For $n = 64$ sorted elements, state the maximum number of comparisons binary search requires (using $\log_2 64$), and compare it to linear search's worst case of 64 comparisons.
6. Write the recurrence relation $T(n)$ for the recursive binary search, and show, by unrolling one or two levels, why $T(n) = T(n/2) + O(1)$ leads to $O(\log n)$ total calls.

References and Notes on Coverage

- **M. T. Goodrich**, *Data Structures and Algorithms in Java* and **T. A. Standish**, *Data Structures in Java* – primary sources for the sorting algorithms (selection sort, insertion sort), their complexity analysis, and the linear/binary search algorithms in Sections 3.3–3.4; the pattern of comparing best/average/worst-case behaviour follows their treatment closely, adapted here from Java to C.
- **R. G. Dromey**, *How to Solve it by Computer* – reference for decomposing the sorting and searching problems methodically, and for the general problem-solving framing of Section 3.1.
- **D. Gries**, *The Science of Programming* – underlies the recursion-induction style of reasoning about recursive-function correctness discussed in Section 3.2.
- **O. J. Dahl**, **E. W. Dijkstra**, **C. A. R. Hoare**, *Structured Programming* and **E. W. Dijkstra**, *A Short Introduction to the Art of Programming* – underlie the functional-decomposition motivation in Section 3.1 and the disciplined treatment of recursive procedures.

Coverage note: as in Chapter 2, the C-specific syntax for function prototypes, pointer-based pass-by-reference, and `strcmp`-based string comparison is not contained in the six reference books (which are language-methodology texts, or – for Goodrich/Standish – Java-based). These notes adapt the underlying algorithmic and methodological content of those books into C; for C-specific syntax reference, Kernighan & Ritchie's *The C Programming Language* remains the recommended supplementary text.

Chapter 4

Datatypes in C

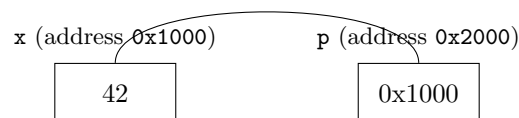
1. Pointers: Pointer variables. Declaring and dereferencing pointer variables. Pointer Arithmetic. Examples. Accessing arrays through pointers. Pointer types, Pointers and strings. String operations in C.
2. Structures in C: Motivation, examples, declaration, and use. Operations on structures. Passing structures as function arguments. type defining structures.
3. Self-referential structures. Dynamic Data Structures. Linked Lists. Examples
4. File input-output in C. Streams. Input, output and error streams. Opening, closing and reading from files. Programming for command line arguments

4.1 Pointers

4.1.1 Pointer Variables: Declaration and Dereferencing

[Pointer] A *pointer* is a variable whose value is the memory *address* of another variable, rather than an ordinary data value. A pointer's declared type indicates the type of the object it points *to*.

```
1 int x = 42;
2 int *p;      // p is declared as "pointer to int"
3 p = &x;      // & is the address-of operator: p now holds x's address
4 printf("%d\n", *p); // * is the dereference operator: prints 42 (x's value)
5 *p = 100;    // modifies x indirectly, through the pointer
6 printf("%d\n", x); // prints 100
```



The diagram shows `p` storing the address of `x`; dereferencing `p` (`*p`) follows the arrow back to `x`'s actual storage location – exactly the mechanism used to achieve pass-by-reference in Chapter 3.

[Pointer to Pointer]

```
1 int x = 5;
2 int *p = &x;
3 int **pp = &p; // pp holds the address of p itself
4 printf("%d\n", **pp); // dereference twice: pp -> p -> x, prints 5
```

Just as `p` points to `x`, `pp` points to `p`; `**pp` dereferences twice to reach `x`'s value. Multi-level pointers arise naturally when a function needs to modify a caller's pointer variable itself (not just the value it points to).

4.1.2 Pointer Arithmetic

Unlike ordinary integers, arithmetic on a pointer is automatically scaled by the size of the type it points to.

[Pointer Arithmetic] If `p` is a pointer of type `T*`, then `p + 1` yields the address `sizeof(T)` bytes after `p`, not simply one byte after – the compiler multiplies the integer offset by `sizeof(T)` automatically.

[Pointer Arithmetic on an Integer Array]

```

1 int arr[4] = {10, 20, 30, 40};
2 int *p = arr;           // p points to arr[0]; array name decays to a pointer
3
4 printf("%d\n", *p);      // 10  (arr[0])
5 printf("%d\n", *(p + 1)); // 20  (arr[1]) -- advances by sizeof(int) = 4 bytes
6 printf("%d\n", *(p + 2)); // 30  (arr[2])
7 p++;                    // p now points to arr[1]
8 printf("%d\n", *p);      // 20

```

0x1000	0x1004	0x1008	0x100C
10	20	30	40
p / p+0	p+1	p+2	p+3

Each step of `p+1` advances by 4 bytes (`sizeof(int)`), not 1 byte – this is why pointer arithmetic “just works” for indexing into arrays of any element type, and why `arr[i]` is defined to mean exactly `*(arr + i)`.

4.1.3 Accessing Arrays Through Pointers

[Array Indexing vs. Pointer Dereferencing – Equivalent Forms]

```

1 int arr[5] = {1, 2, 3, 4, 5};
2 int *p = arr;
3
4 // The following pairs are exactly equivalent in C:
5 arr[2]      == *(arr + 2)  == *(p + 2)  == p[2]
6
7 // Summing an array using pointer arithmetic instead of indexing:
8 int sum = 0;
9 for (int *q = arr; q < arr + 5; q++) {
10     sum += *q;
11 }
12 printf("Sum = %d\n", sum);    // 15

```

The loop above walks a pointer `q` from `arr` (the address of `arr[0]`) up to (but not including) `arr + 5` (one past the last element) – a common idiom in C, and conceptually identical to an index-based `for` loop but expressed purely in terms of addresses.

4.1.4 Pointer Types

Every pointer has a declared type, which determines both the size of steps in pointer arithmetic and what dereferencing yields.

Declaration	Meaning
<code>int *p</code>	pointer to <code>int</code>
<code>char *s</code>	pointer to <code>char</code> (commonly used for strings)
<code>double *d</code>	pointer to <code>double</code>
<code>int **pp</code>	pointer to pointer to <code>int</code>
<code>void *vp</code>	generic pointer – points to memory of unspecified type; cannot be dereferenced directly, must be cast first

[Why Pointer Type Matters]

```

1 int x = 65;
2 int *pi = &x;
3 char *pc = (char *) &x;    // reinterpret the same address as a char pointer
4
5 printf("%d\n", *pi);    // 65 -- reads all 4 bytes of x as an int
6 printf("%d\n", *pc);    // 65 (on a little-endian system) -- reads only the FIRST
    byte

```

Both `pi` and `pc` hold the *same address*, but `*pi` reads `sizeof(int)` bytes while `*pc` reads only `sizeof(char)` = 1 byte – directly connecting to the endianness discussion in Chapter 2, since which byte `*pc` reads depends on the system’s byte ordering.

4.1.5 Pointers and Strings; String Operations in C

A C string is conventionally accessed via a `char *` pointing to the first character of a null-terminated character sequence (Chapter 2, Section on Arrays and Strings).

[String Literals and Pointers]

```

1 char *greeting = "Hello";    // greeting points to the first character 'H'
2 printf("%c\n", *greeting);    // 'H'
3 printf("%c\n", *(greeting+1)); // 'e'
4 printf("%s\n", greeting);    // "Hello" -- %s prints until '\0'

```

[Writing `strlen` from Scratch Using Pointers]

```

1 int myStrlen(const char *s) {
2     int count = 0;
3     while (*s != '\0') {
4         count++;
5         s++;
6     }
7     return count;
8 }
9 // myStrlen("Hello") returns 5

```

This mirrors how the standard library’s `strlen` is typically implemented: walk a pointer forward until the null terminator is reached, counting steps – a direct pointer-arithmetic analogue of the array-indexing version a beginner might write first.

[Common String Operations from `<string.h>`]

```

1 #include <string.h>
2 char a[20] = "Hello";
3 char b[20] = "World";
4
5 printf("%lu\n", strlen(a));    // 5
6 strcat(a, b);                // a becomes "HelloWorld"
7 strcpy(a, "Reset");          // a becomes "Reset" (overwritten)
8 printf("%d\n", strcmp("apple", "banana")); // negative (since 'a' < 'b')
9 printf("%d\n", strcmp("cat", "cat"));    // 0 (equal)

```

Assignments

1. Given `int y = 17; int *q = &y;`, write the statement that changes `y` to 34 using only `q` (not `y` directly), and state the value printed by `printf("%d", y)` afterward.
2. Given `int arr[5] = {2, 4, 6, 8, 10};` and `int *p = arr;`, evaluate `*(p+3)`, `p[1]`, and `*(arr+4)` by hand.
3. If `double *d` points to address `0x3000` and `sizeof(double) = 8`, what address does `d + 3` hold?
4. Trace `myStrlen("Recursion")` by hand, listing the value of `count` after each iteration, and confirm the final result is 9.
5. Evaluate `strcmp("apple", "apple")`, `strcmp("dog", "cat")`, and `strcmp("cat", "dog")`, stating whether each result is negative, zero, or positive (exact numeric values are implementation-defined, but the sign is not).
6. Given `char *s = "Programming";`, write a pointer-based loop (in the style of `myStrlen`) that counts how many times the letter `'r'` occurs, and trace it by hand, confirming the count is 2.

4.2 Structures in C

4.2.1 Motivation

Arrays group multiple values of the *same* type under one name. Many real-world entities, however, combine *different* types of related data – e.g. a student record needs a name (string), roll number (int), and GPA (float) together. A *structure* lets a programmer define a new composite type bundling heterogeneous fields, echoing the modular decomposition philosophy from Dromey’s *How to Solve it by Computer* applied to *data* rather than just control flow.

4.2.2 Declaration and Use

```

1 struct Student {
2     char name[30];
3     int rollNumber;
4     float gpa;
5 };
6
7 int main(void) {
8     struct Student s1;           // declare a variable of type struct Student
9     strcpy(s1.name, "Alice");    // '.' accesses a member (field)
10    s1.rollNumber = 101;
11    s1.gpa = 8.7;
12
13    printf("%s, Roll: %d, GPA: %.1f\n", s1.name, s1.rollNumber, s1.gpa);
14    return 0;
15 }
```

[Initializing a Structure at Declaration]

```

1 struct Student s2 = {"Bob", 102, 7.9};    // members initialized in declared order
2 printf("%s\n", s2.name);    // "Bob"
```

4.2.3 Operations on Structures

[Structure Assignment Copies All Members]

```

1 struct Student s3 = s1;    // copies EVERY field of s1 into s3 (a deep, member-wise
   copy
2                               // for non-pointer fields)
3 s3.rollNumber = 999;       // modifying s3 does NOT affect s1
4 printf("%d %d\n", s1.rollNumber, s3.rollNumber);    // 101 999

```

Unlike arrays (which decay to pointers and cannot be assigned wholesale with =), C structures *can* be assigned directly, and the assignment copies each member individually – this is a genuine value copy, not a reference.

[Nested Structures]

```

1 struct Date {
2     int day, month, year;
3 };
4 struct Student {
5     char name[30];
6     struct Date dob;    // a structure containing another structure
7 };
8
9 struct Student s;
10 s.dob.day = 15;
11 s.dob.month = 8;
12 s.dob.year = 2005;
13 printf("%d/%d/%d\n", s.dob.day, s.dob.month, s.dob.year);

```

The chained . operator (`s.dob.day`) accesses a field of a field – structures can nest to arbitrary depth, modelling hierarchical data naturally.

4.2.4 Passing Structures as Function Arguments

[Pass by Value – A Full Copy Is Made]

```

1 void printStudent(struct Student s) {    // entire structure copied onto the stack
2     printf("%s\n", s.name);
3 }

```

For large structures, passing by value can be costly (the whole structure is copied); this parallels the pass-by-value discussion in Chapter 3, now at the granularity of an entire record rather than a single number.

[Pass by Reference – Avoiding the Copy, and Allowing Modification]

```

1 void updateGpa(struct Student *s, float newGpa) {
2     s->gpa = newGpa;    // -> dereferences a structure pointer and accesses a
   member
3                               // equivalent to (*s).gpa = newGpa;
4 }
5
6 int main(void) {
7     struct Student s1 = {"Alice", 101, 8.7};
8     updateGpa(&s1, 9.2);
9     printf("%.1f\n", s1.gpa);    // 9.2 -- the caller's structure was modified
10    return 0;
11 }

```

The `->` operator is shorthand for dereferencing a pointer to a structure and then accessing a member in one step; `s->gpa` is exactly equivalent to `(*s).gpa`. Passing a pointer avoids copying the whole structure and, as with arrays and simple pass-by-reference in Chapter 3, allows the function to modify the caller's original data.

4.2.5 Type-Defining Structures with typedef

[typedef for a Cleaner Structure Type Name]

```

1 typedef struct {
2     char name[30];
3     int rollNumber;
4     float gpa;
5 } Student; // "Student" is now a type name -- no need to write "struct Student"
6
7 Student s1 = {"Carol", 103, 9.0}; // no "struct" keyword needed
8 Student *ptr = &s1;
9 printf("%s\n", ptr->name);

```

`typedef` does not create a new kind of structure; it simply gives the (possibly anonymous) structure type a shorter alias, which is why almost all production C code defines structures this way rather than repeatedly writing `struct TagName`.

Assignments

1. Define a `struct Point` with integer members `x` and `y`. Declare two variables `p1 = {3, 4}` and `p2 = {0, 0}`, and write a function `double distance(struct Point a, struct Point b)` that returns the Euclidean distance between them; compute the expected numeric result for `p1` and `p2` by hand (5.0).
2. Given the `struct Student` assignment example, if `s1.rollNumber = 101` and, after `s3 = s1;`, we execute `s1.rollNumber = 555;`, what value does `s3.rollNumber` hold? Justify using the copy semantics of structure assignment.
3. Rewrite `updateGpa` so that it takes the structure *by value* instead of by pointer, and explain (referring to Chapter 3's pass-by-value rules) why the caller's GPA would then remain unchanged after the call.
4. Using `typedef struct { int day, month, year; } Date;`, declare a variable `Date today = {10, 7, 2026};` and write the `printf` statement to print it as 10/7/2026.
5. Define a nested structure `struct Rectangle` containing two `struct Point` corners (top-left and bottom-right), and write a function that computes its area, tracing it by hand for corners (0,0) and (4,3) (expected area: 12).

4.3 Self-Referential Structures and Dynamic Data Structures

4.3.1 Self-Referential Structures

[Self-Referential Structure] A *self-referential structure* is a structure that contains, among its members, a pointer to another structure of the *same* type. This is what allows structures to be linked together into chains, trees, and other dynamic structures at run time.

```

1 struct Node {
2     int data;
3     struct Node *next; // pointer to another struct Node -- the self-reference
4 };

```

Note that `struct Node` cannot contain a member of type `struct Node` directly (that would require infinite memory), but *can* contain a `struct Node *`, since a pointer has a fixed, known size regardless of what it points to.

4.3.2 Dynamic Memory Allocation

Self-referential structures are typically combined with *dynamic memory allocation* (via `malloc` from `<stdlib.h>`), which requests memory from the heap at run time, whose size need not be known when the program is compiled.

[Allocating and Freeing a Single Node]

```

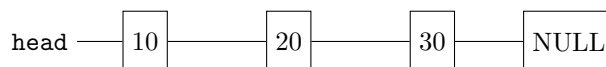
1 #include <stdlib.h>
2
3 struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
4 if (newNode == NULL) {
5     // allocation failed -- always check!
6 }
7 newNode->data = 10;
8 newNode->next = NULL;
9 // ... use newNode ...
10 free(newNode); // release the memory back to the system when done

```

`malloc(sizeof(struct Node))` requests enough bytes to hold one `struct Node` and returns a `void*`, which is cast to `struct Node*`. Every successful `malloc` should eventually be matched with a `free` to avoid a *memory leak*.

4.3.3 Linked Lists

[Linked List] A *linked list* is a linear dynamic data structure consisting of a sequence of nodes, where each node holds data plus a pointer to the next node in the sequence; the list itself is represented by a pointer (**head**) to the first node, and the last node's **next** pointer is `NULL`.



Each box's left half is the **data** field; the right half is the **next** pointer, drawn as an arrow to the following node (or to `NULL` at the end) – this is the standard diagram for reasoning about linked-list operations on the board.

[Inserting a Node at the Head]

```

1 struct Node *insertAtHead(struct Node *head, int value) {
2     struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
3     newNode->data = value;
4     newNode->next = head; // new node points to the old head
5     return newNode;      // new node becomes the new head
6 }
7 // usage: head = insertAtHead(head, 5);

```

[Traversing and Printing a Linked List]

```

1 void printList(struct Node *head) {
2     struct Node *current = head;
3     while (current != NULL) {
4         printf("%d -> ", current->data);
5         current = current->next;
6     }
7     printf("NULL\n");
8 }

```

This traversal is structurally identical to the iterative linear search of Chapter 3 (a `while` loop advancing a pointer until it hits a sentinel value) – only here the “next element” comes from a **next** pointer rather than array indexing.

[Searching a Linked List]

```

1 struct Node *search(struct Node *head, int target) {
2     struct Node *current = head;
3     while (current != NULL) {
4         if (current->data == target)
5             return current;    // found -- return pointer to the node
6         current = current->next;
7     }
8     return NULL;              // not found
9 }

```

Because a linked list provides no random access (unlike an array), only *linear* search is possible here – binary search, which relies on jumping directly to a middle index, cannot be applied to a plain linked list.

[Deleting a Node by Value]

```

1 struct Node *deleteNode(struct Node *head, int value) {
2     struct Node dummy;
3     dummy.next = head;
4     struct Node *prev = &dummy;
5     struct Node *current = head;
6
7     while (current != NULL) {
8         if (current->data == value) {
9             prev->next = current->next;    // unlink current
10            free(current);
11            break;
12        }
13        prev = current;
14        current = current->next;
15    }
16    return dummy.next;
17 }

```

The `dummy` node (whose `next` initially points to `head`) is a common trick that avoids special-casing a deletion at the head of the list – `prev` always has a valid predecessor node to update, even when deleting the very first element.

Assignments

1. Starting from an empty list (`head = NULL`), trace the sequence of calls `head = insertAtHead(head, 3);`, `head = insertAtHead(head, 7);`, `head = insertAtHead(head, 1);`, drawing the resulting linked list (in the style of the diagram above) after each call.
2. For the list $1 \rightarrow 7 \rightarrow 3 \rightarrow \text{NULL}$ from the previous question, trace `search(head, 7)`, listing which nodes are visited before it returns.
3. Trace `deleteNode(head, 7)` on the same list, describing exactly which `next` pointer is updated, and draw the resulting list.
4. Write a function `int listLength(struct Node *head)` that counts the number of nodes in a linked list, and trace it by hand for the 3-node list above, confirming the count is 3.
5. Explain, referencing the definition of a self-referential structure, why `struct Node { int data; struct Node next; }` (without the `*`) would not compile, while `struct Node { int data; struct Node *next; }` does.

4.4 File Input-Output in C

4.4.1 Streams: Input, Output, and Error

[Stream] A *stream* is an abstraction representing a sequence of bytes flowing between a program and a source or destination (a file, the keyboard, the screen, etc.), accessed uniformly through the `FILE*` type regardless of the underlying device.

Every C program automatically has three streams open at startup:

Stream	Purpose	Default destination
<code>stdin</code>	Standard input	Keyboard
<code>stdout</code>	Standard output	Screen (terminal)
<code>stderr</code>	Standard error	Screen (terminal), unbuffered

[`stdout` vs. `stderr`]

```
1 fprintf(stdout, "Processing complete.\n");    // same as printf(...)
2 fprintf(stderr, "Warning: input file not found.\n");
```

Separating `stdout` from `stderr` allows shell redirection to route them independently, e.g. `./prog > output.txt` sends only `stdout` to the file while error messages still appear on the terminal – connecting directly to the redirection operators (`>`, `>>`) covered in the Linux/shell section of Chapter 1.

4.4.2 Opening, Closing, and Reading Files

Mode	Meaning
" <code>r</code> "	Read (file must already exist)
" <code>w</code> "	Write (creates new file, or truncates an existing one)
" <code>a</code> "	Append (writes are added to the end of an existing file)
" <code>r+</code> "	Read and write (file must exist)

[Writing to a File]

```
1 #include <stdio.h>
2
3 FILE *fp = fopen("scores.txt", "w");
4 if (fp == NULL) {
5     fprintf(stderr, "Error: could not open file.\n");
6     return 1;
7 }
8 fprintf(fp, "Alice %d\n", 90);
9 fprintf(fp, "Bob %d\n", 85);
10 fclose(fp);    // always close a file when done -- flushes buffered output
```

[Reading a File Line by Line]

```
1 FILE *fp = fopen("scores.txt", "r");
2 if (fp == NULL) {
3     fprintf(stderr, "Error: could not open file.\n");
4     return 1;
5 }
6 char name[30];
7 int score;
8 while (fscanf(fp, "%s %d", name, &score) == 2) {
9     printf("%s scored %d\n", name, score);
10 }
11 fclose(fp);
```

`fscanf` returns the number of items successfully read; comparing it to 2 (the number of conversion specifiers) is a standard way to detect the end of the file or a malformed line, functioning as the loop's termination condition.

[Reading an Entire File with `fgets`]

```

1 FILE *fp = fopen("notes.txt", "r");
2 char line[256];
3 while (fgets(line, sizeof(line), fp) != NULL) {
4     printf("%s", line);    // fgets keeps the trailing newline, if present
5 }
6 fclose(fp);

```

`fgets` is generally safer than `fscanf("%s", ...)` for reading whole lines (including spaces), since it takes an explicit buffer size and will not overflow the destination array, addressing the buffer-overflow risk highlighted for arrays in Chapter 2.

4.4.3 Programming for Command-Line Arguments

[Command-Line Arguments] A C program's `main` function may be declared to receive the number of command-line arguments (`argc`) and an array of pointers to the argument strings themselves (`argv`), allowing behaviour to be controlled from outside the program at invocation time.

```

1 int main(int argc, char *argv[]) {
2     printf("Program name: %s\n", argv[0]);
3     printf("Number of arguments: %d\n", argc);
4     for (int i = 1; i < argc; i++) {
5         printf("Argument %d: %s\n", i, argv[i]);
6     }
7     return 0;
8 }

```

[Running with Arguments] Invoking the program above from the shell as:

```

1 $ ./myprog data.txt 42 verbose

```

sets `argc = 4` and `argv = {"/myprog", "data.txt", "42", "verbose"}` (indices 0 through 3), so `argv[0]` is always the program's own invocation name, and `argv[1]` through `argv[argc-1]` are the actual arguments supplied by the user – directly connecting to the shell-scripting material in Chapter 1's Linux tools section.

[A File-Processing Program Driven by Command-Line Arguments]

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[]) {
5     if (argc != 2) {
6         fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
7         return 1;
8     }
9     FILE *fp = fopen(argv[1], "r");
10    if (fp == NULL) {
11        fprintf(stderr, "Error: cannot open %s\n", argv[1]);
12        return 1;
13    }
14    int count = 0, ch;
15    while ((ch = fgetc(fp)) != EOF) {
16        if (ch == '\n') count++;
17    }
18    printf("%s has %d lines.\n", argv[1], count);
19    fclose(fp);
20    return 0;

```



```
21 }  
22 // usage: ./countlines report.txt
```

This program combines every idea from this section: it validates `argc` before use, opens a file named by `argv[1]`, reports errors on `stderr`, reads the file character by character with `fgetc` until EOF, and closes the file when finished.

Assignments

1. Explain the difference between opening a file with mode `"w"` versus `"a"`, and predict the contents of `log.txt` after running `fopen("log.txt", "w")`, writing `"first"`, closing it, then `fopen("log.txt", "a")`, writing `"second"`, and closing it again.
2. Write C code that opens `numbers.txt` for reading, reads integers one at a time using `fscanf`, and prints their sum, stopping when `fscanf` fails to read a value. Trace it by hand assuming the file contains `4 8 15 16 23` (expected sum: 66).
3. Explain, using the definition of streams, why `fprintf(stderr, ...)` messages still appear on the terminal even when a program is run as `./prog > output.txt`.
4. If a program is invoked as `./analyze data1.csv data2.csv -verbose`, state the values of `argc` and each element of `argv`.
5. Modify the line-counting program so that it also counts the total number of characters in the file (including newlines), and trace its output by hand for a 3-line file where each line has 10 characters plus a newline (expected: 3 lines, 33 characters).

References and Notes on Coverage

- **M. T. Goodrich, R. Tamassia, M. H. Goldwasser, *Data Structures and Algorithms in Java*** and **T. A. Standish, *Data Structures in Java*** – primary sources for the *conceptual* treatment of linked lists in Section 4.3 (node structure, traversal, insertion, deletion), though both present these ideas using Java references rather than C's explicit pointers and `malloc/free`.
- **R. G. Dromey, *How to Solve it by Computer*** – underlies the motivation for structures in Section 4.2 as a tool for decomposing a problem's data, paralleling the decomposition of a problem's control flow into functions (Chapter 3).
- **O. J. Dahl, E. W. Dijkstra, C. A. R. Hoare, *Structured Programming*** – underlies the general principle, reflected throughout this chapter, of building complex data organizations from simple, well-defined building blocks.

Coverage note: pointers, `malloc/free`-based manual memory management, C structures with `->`, and C's stream-based file I/O (`FILE*`, `fopen/fclose`, `argc/argv`) are C-specific mechanisms not present in Goodrich or Standish (Java manages memory automatically via garbage collection and has no raw pointers) nor in the remaining reference books (which are language-agnostic). As in previous chapters, Kernighan & Ritchie's *The C Programming Language* is recommended as the primary supplementary reference for this chapter's C-specific syntax and standard-library usage.

Chapter 5

Previous Year Question Paper with Solutions

/

5.0.1 Midterm

Fundamentals of Computing and Programming

Mid Term Examination. 50 marks, Time Limit 2 hrs

October 14, 2022

1. **Rewrite the code below** using a single **switch** instead of using **if**: [3]

```
void f(int x){
    if (x == 2 )
        f();
    if ( x == 2 || x == 3 )
        common();
    else
        h();
}
```

2. **Rewrite this code below** using **for** instead of **do ... while**: [3]

```
void g(int i){
    do {
        printf( "%d \n", i);
        i--;
    } while( i > 0 );
}
```

3. This question is with respect to a function called **quiz**:
- (a) **Declare the prototype** of the function given it takes **two parameters**- the first is a character **c** and the second is an integer **n**. It **returns a character**. [1]

- (b) The body contains **exactly one line**, it returns the character that is `n` positions after the character in `c`. **Write out the function.**

[2]

4. **Write a function** named `myrange` to take two integer **parameter** values `x` and `y`. It prints all integers greater than `x` but less than `y` in decreasing order. It normally returns 0; however, if the range is empty, it returns -1.

[3]

5. Answer in one line for each of these:

[8]

- (a) Differentiate C source code from a compiled executable code.
- (b) What does a `#include` directive do?
- (c) What is the difference between the directive `#define x 2` and the statement `int x 2`;
- (d) How does a parameter to a function get its value?
- (e) To be useful a recursive function always needs a condition check. Why?
- (f) Why do we need the ampersand sign in this use of `scanf`:
`int x; scanf("%d",&x)`
- (g) What does the following print and why:
`float f = 5 / 2 ; printf("%f", f)`
- (h) Assuming `a` and `b` are integer variables, which are already initialized, what do the following statements achieve:

```
a=a+b;
b=a-b;
a=a-b;
```

6. What does the following code print:

[3]

```
if (2 < 1 < 3)
    printf("1: True\n");
if (1 > 0 == 0)
    printf("2: True\n");
if (1 > 2 <= 0)
    printf("3: True\n");
```

7. Assume there is a function `random1000()` which returns a random integer in the range $[0 \dots 1000]$. **Write a while loop** that repeatedly iterates as long as `random1000()` returns a value less than 500. Inside the loop it reads another integer from the user. If that integer is negative it immediately terminates the loop. [3]
8. **Write a function** `void swap(int *p1, int *p2)` which swaps the integers pointed to by its two parameters. **Also write a main()** function that calls `swap` with appropriate parameters to illustrate how the function `swap` works. [3]
9. **Write a function** `print_triangle(n)` that prints a pattern of the following kind of shape depending on the parameter value: [6]

```
* * * * *
  * * *
    *
```

The first row always has $2*n-1$ stars and the bottom row has one star. The above illustration is the result of the call `print_triangle(3)`; Note that in each row the stars are blank separated. Note how the stars are aligned. As an idea note that the first row has zero spaces at the beginning, then 5 stars separated by blanks. The second row has two spaces at the beginning then 3 stars separated by blanks. The third row has four spaces at the beginning then 1 star. You may assume that the parameter given will always be positive and it indicates the number of rows of stars to print.

10. **Write a function** `reverse(n, array1, array2)`. The parameter `n` is the number of elements in `array1`. It copies the elements of `array1` into `array2`, but in reverse order. Assume the arrays are integer arrays. [3]
11. An array has n integers. A recursive algorithm to sum the elements is:
 $sum(a[0] \dots a[n-1]) = a[0] + sum(a[1] \dots a[n-1])$.
Write a recursive C function called `sum_array` using this idea. Write a main function that initializes an array to 10 integers and calls your function to compute and then print the sum. [6]

12. **Write a function** `insert(arr, n, val, pos)` which will insert an integer `val` into the integer array `arr` at the index `pos`. The parameter `n` is the number of elements currently in the array. As an example if the array contains { 8 4 7 1 2 } and we call `insert(arr, 5, 9, 2)`, after the call returns the array will be { 8 4 9 7 1 2 }. [6]

Fundamentals of Computing and Programming

Midterm Examination – Solutions

(Original exam: October 14, 2022; 50 marks, 2 hr time limit)

Question 1 (3 marks)

Rewrite the given code using a single switch instead of if.

Original code:

```
1 void f(int x){
2     if (x == 2)
3         f();
4     if (x == 2 || x == 3)
5         common();
6     else
7         h();
8 }
```

Reasoning: The two if statements are *independent* (the second is not an else of the first), so both must be evaluated for every value of x :

- $x = 2$: first if is true (calls the inner function), and the second condition ($x==2 \ || \ x==3$) is also true, so `common()` is also called.
- $x = 3$: first if is false, second condition is true, so only `common()` is called.
- any other x : first if is false, second condition is false, so only `h()` is called.

A single `switch` reproduces this by letting the $x = 2$ case *fall through* into the $x = 3$ case's action:

```
1 void f(int x){
2     switch (x) {
3         case 2:
4             f();           // extra action only for x == 2
5             common();      // falls through into the x == 3 case's
6                             action
7             break;
8         case 3:
9             common();
10            break;
11        default:
12            h();
13    }
```

Note: the call `f()`; inside `f(int x)` in the original code is a same-name call with no arguments, which as literally written would not compile (it does not match the declared signature `f(int)`). This is preserved verbatim from the question as given; in an actual answer, assume it represents *some* function call the exact identity of which does not affect the required control-flow transformation.

Question 2 (3 marks)

Rewrite the code below using `for` instead of `do...while`.

```
1 void g(int i){
2     do {
3         printf("%d \n", i);
4         i--;
5     } while (i > 0);
6 }
```

Direct translation (the expected answer for this exam):

```
1 void g(int i){
2     for ( ; i > 0; i--) {
3         printf("%d \n", i);
4     }
5 }
```

Subtlety worth noting when grading: a `do...while` loop always executes its body *at least once*, even if the condition is false from the start, whereas a `for` loop (like a plain `while`) tests its condition *before* the first iteration. The translation above therefore differs from the original when `g` is called with `i <= 0`: the original prints `i` once and stops, while the `for` version above prints nothing. A fully equivalent version for all inputs would be:

```
1 void g(int i){
2     int first = 1;
3     for ( ; first || i > 0; i--) {
4         printf("%d \n", i);
5         first = 0;
6     }
7 }
```

For grading purposes, the simple translation is the expected answer for 3 marks; the refinement above is worth mentioning as bonus insight or partial-credit discussion.

Question 3

Function quiz: returns the character `n` positions after character `c`.

(a) **Prototype** [1 mark]:

```
1 char quiz(char c, int n);
```

(b) **Definition** [2 marks]:

```
1 char quiz(char c, int n) {
2     return (char)(c + n);
3 }
```

Explanation: in C, `char` values are promoted to `int` for arithmetic, so `c + n` computes the correct numeric character code `n` positions after `c` (relying on the underlying character encoding, e.g. ASCII, being contiguous over the relevant range); the explicit cast back to `char` on return matches the declared return type. For example, `quiz('a', 3)` returns `'d'`.

Question 4 (3 marks)

`myrange(x, y)`: print all integers strictly between `x` and `y` in decreasing order; return 0 normally, `-1` if the range is empty.

```

1 int myrange(int x, int y) {
2     if (y - x <= 1) {          // no integer i satisfies x < i < y
3         return -1;
4     }
5     for (int i = y - 1; i > x; i--) {
6         printf("%d\n", i);
7     }
8     return 0;
9 }

```

Explanation: the range (x, y) contains no integers exactly when $y \leq x + 1$ (e.g. $x = 3, y = 4$ has no integer strictly between them, and neither does $y \leq x$). The loop starts at the largest qualifying integer, $y - 1$, and counts down to (but not including) x , printing in decreasing order as required.

Question 5 (8 marks – one line each)

- (a) **C source code vs. compiled executable:** C source code is human-readable text written according to C syntax (a `.c` file) that must be translated by a compiler before it can run; the compiled executable is machine code (binary instructions) produced from that source, which the CPU/operating system can run directly.
- (b) **#include directive:** it instructs the preprocessor to textually insert the entire contents of the named file (typically declarations and macros from a header) at that point in the source, before compilation proper begins.
- (c) **#define x 2 vs. int x = 2;:** `#define x 2` is a preprocessor macro that performs a purely textual substitution of `x` with `2` everywhere in the code before compilation – `x` is not a variable, occupies no memory, and cannot change at run time. `int x = 2;` declares an actual variable `x` with allocated memory, an `int` type, and a run-time value that can subsequently be modified.
- (d) **How a parameter gets its value:** it is initialized from the value of the corresponding argument expression supplied by the caller at the point of the function call; in C, this is always pass-by-value, i.e. the argument's value is *copied* into the parameter.
- (e) **Why recursion needs a condition check:** without a base-case condition that stops further recursive calls, the function would call itself indefinitely, causing unbounded growth of the call stack and eventually a stack overflow/crash; the condition check provides the terminating base case.
- (f) **Why & is needed in scanf("%d", &x):** `scanf` needs the *address* of `x` so it can write the input value directly into `x`'s memory location; since C passes arguments by value, passing `x` itself would only give `scanf` a copy of its current value, leaving it no way to modify the caller's variable.
- (g) **float f = 5 / 2; printf("%f", f);:** this prints 2.000000, not 2.500000 – because 5 and 2 are both `int` constants, `5 / 2` is evaluated using *integer* division first (yielding 2, with the remainder discarded), and only *afterward* is that integer result converted to `float` for storage in `f`.
- (h) **a=a+b; b=a-b; a=a-b;:** this is the classic no-temporary-variable swap: after these three statements execute, `a` and `b` end up holding each other's original values (i.e. `a` and `b` are swapped), computed purely through arithmetic rather than using a third variable.

Question 6 (3 marks)

What does the following code print?

```
1 if (2 < 1 < 3)
2     printf("1: True\n");
3 if (1 > 0 == 0)
4     printf("2: True\n");
5 if (1 > 2 <= 0)
6     printf("3: True\n");
```

Output:

```
1 1: True
2 3: True
```

Explanation: C has no concept of mathematical chained comparisons; each expression is evaluated strictly left-to-right according to operator precedence and associativity, with relational/equality operators producing 0 or 1:

- $2 < 1 < 3$: first $(2 < 1)$ is 0, then $0 < 3$ is 1 (true) $\Rightarrow$ prints line 1.
- $1 > 0 == 0$: $>$ binds tighter than $==$, so this is $(1 > 0) == 0$, i.e. $1 == 0$, which is 0 (false) $\Rightarrow$ does not print.
- $1 > 2 <= 0$: $>$ and $<=$ have equal precedence, left-to-right, so this is $(1 > 2) <= 0$, i.e. $0 <= 0$, which is 1 (true) $\Rightarrow$ prints line 3.

This question is an excellent teaching moment for operator precedence and the absence of chained comparisons in C.

Question 7 (3 marks)

Loop that continues while `random1000()` returns less than 500; inside, reads an integer and terminates immediately if it is negative.

```
1 int val;
2 while (random1000() < 500) {
3     scanf("%d", &val);
4     if (val < 0) {
5         break;
6     }
7 }
```

Explanation: the `while` condition re-evaluates `random1000()` at the start of every iteration (a fresh random number each time). Inside the loop, an integer is read; if it is negative, `break` exits the loop immediately, overriding the outer condition.

Question 8 (3 marks)

`void swap(int *p1, int *p2)` that swaps the integers pointed to by its parameters, plus a demonstrating main.

```
1 #include <stdio.h>
2
3 void swap(int *p1, int *p2) {
4     int temp = *p1;
5     *p1 = *p2;
```

```

6     *p2 = temp;
7 }
8
9 int main(void) {
10     int a = 5, b = 10;
11     printf("Before swap: a = %d, b = %d\n", a, b);
12     swap(&a, &b);
13     printf("After swap:  a = %d, b = %d\n", a, b);
14     return 0;
15 }

```

Expected output:

```

1 Before swap: a = 5, b = 10
2 After swap:  a = 10, b = 5

```

Explanation: passing `&a` and `&b` gives `swap` the *addresses* of the caller's variables, so dereferencing (`*p1`, `*p2`) inside the function reads and writes `a` and `b` directly – this is how C achieves the effect of pass-by-reference.

Question 9 (6 marks)

`print_triangle(n)`: prints a triangle of blank-separated stars, $2n - 1$ stars on the first row, decreasing by 2 each row down to 1 star on the last row, with increasing leading indentation.

Deriving the pattern for row i ($i = 1, \dots, n$, counting from the top):

$$\text{stars in row } i = 2(n - i) + 1, \quad \text{leading spaces in row } i = 2(i - 1).$$

Check against the example for $n = 3$: row 1 has $2(3 - 1) + 1 = 5$ stars and 0 leading spaces; row 2 has $2(3 - 2) + 1 = 3$ stars and 2 leading spaces; row 3 has $2(3 - 3) + 1 = 1$ star and 4 leading spaces – matching the given illustration exactly.

```

1 void print_triangle(int n) {
2     for (int i = 1; i <= n; i++) {
3         int spaces = 2 * (i - 1);
4         int stars  = 2 * (n - i) + 1;
5
6         for (int s = 0; s < spaces; s++) {
7             printf(" ");
8         }
9         for (int k = 0; k < stars; k++) {
10            printf("*");
11            if (k < stars - 1) {
12                printf(" ");    // blank-separate stars, no trailing
                                space
13            }
14        }
15        printf("\n");
16    }
17 }

```

Trace for `print_triangle(3)`:

```

1 * * * * *
2  * * *
3   *

```

Question 10 (3 marks)

`reverse(n, array1, array2)`: copies `array1` into `array2` in reverse order.

```

1 void reverse(int n, int array1[], int array2[]) {
2     for (int i = 0; i < n; i++) {
3         array2[i] = array1[n - 1 - i];
4     }
5 }
```

Explanation: as i runs from 0 to $n - 1$, `array1[n-1-i]` runs from the *last* element of `array1` down to the *first*, so `array2` ends up holding `array1`'s elements in reverse order. For example, `array1 = {1,2,3,4}` produces `array2 = {4,3,2,1}`.

Question 11 (6 marks)

Recursive `sum_array` using $sum(a[0..n-1]) = a[0] + sum(a[1..n-1])$, plus a main that sums a 10-element array.

```

1 #include <stdio.h>
2
3 int sum_array(int a[], int n) {
4     if (n == 0) {
5         return 0; // base case: empty (sub)
6         // array sums to 0
7     }
8     return a[0] + sum_array(a + 1, n - 1); // a+1: pointer to the "rest
9     // of the array
10 }
11
12 int main(void) {
13     int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
14     int total = sum_array(arr, 10);
15     printf("Sum = %d\n", total);
16     return 0;
17 }
```

Expected output: Sum = 55.

Explanation: the base case `n == 0` (an empty array) returns 0, matching the recursive definition's implicit stopping point; the recursive case adds the first element to the sum of the remaining $n - 1$ elements. Passing `a + 1` (pointer arithmetic, Chapter 4) rather than indexing is the idiomatic C way to pass "the rest of the array" to the recursive call.

Question 12 (6 marks)

`insert(arr, n, val, pos)`: inserts `val` into `arr` at index `pos`, where `n` is the current number of elements.

```

1 void insert(int arr[], int n, int val, int pos) {
2     for (int i = n; i > pos; i--) {
3         arr[i] = arr[i - 1]; // shift elements right to make room
4     }
5     arr[pos] = val;
6 }
```

Explanation: to open a gap at index `pos`, every element from the current last index ($n - 1$) up to `pos` must shift one position to the right; the loop does this from the *back* of the array

forward (starting at index `n`, the new last slot) so that no element is overwritten before it has been copied. Once the gap is open, `val` is written into `arr[pos]`.

Trace for `insert(arr, 5, 9, 2)` with `arr = {8, 4, 7, 1, 2}`:

Step	Array state
Start	{8, 4, 7, 1, 2, -}
$i = 5: arr[5] \leftarrow arr[4]$	{8, 4, 7, 1, 2, 2}
$i = 4: arr[4] \leftarrow arr[3]$	{8, 4, 7, 1, 1, 2}
$i = 3: arr[3] \leftarrow arr[2]$	{8, 4, 7, 7, 1, 2}
loop ends ($i = 2$ is not $> pos = 2$)	
$arr[2] \leftarrow 9$	{8, 4, 9, 7, 1, 2}

Final result: {8, 4, 9, 7, 1, 2}, matching the value given in the question. **Note:** this function assumes `arr` has capacity for at least $n + 1$ elements before the call – a point worth making explicit to students, since inserting into a fixed-size array that is already full is undefined behaviour (writes past the end of the array).

Fundamentals of Computing and Programming

Mid Term Examination. Maximum score 40, Total marks 43. Time Limit 2 hrs.

September 21, 2023

1. Answer each of these briefly as instructed:

[4x2=8]

(a) What does the following print:

```
printf("%d %c", 'e' - 'a', 'a'+ 4);  
// prints an integer and a character
```

Explain your answer in one sentence.

(b) Draw a picture and show using arrow for pointer the relationship between the array `a[ ]` and the variables `p` and `q` after the last statement:

```
int a[10]={0,1,2,3,4};  
int *p, *q; // p and q are integer pointers  
p = &a[3];  
q = p-3;
```

(c) What does the following code print:

```
char name[20]="praggnananda";  
printf("%s",name+6);
```

(d) What does the following code print:

```
char name[20]="praggnananda";  
scanf("%s",name+6); // assume the input was "chess"  
printf("%s",name);
```

2. **Downify**

The code below converts given upper case character value to lower case and returns the converted lower case character: [5]

```
char downify(char a){  
    if ( __C__ ) { // check: if a is an upper case character  
                    // then compute the lower case character  
        __S__      // and return it  
    }  
    return a ;     // otherwise just return the given character  
}
```

3. **Array Read/Print**

Write a `main()` function to do the following: [5]

-Define an array named `a` of 10 integers.

- Read **five** integers into the array.
- After all integers are read, then print all the read integers from the array.

4. Array Sum

Write a function `int sum(int a[ ], int n);` [5]
 the first parameter `a[ ]` is an array, the second parameter says how many integers are in the array. The function simply finds the sum of all the n elements of `a[ ]` and returns that value.

5. Flying birds

Write a function called `fly()`. It has one parameter named `s` which is a string. [5]

- It returns 0 or 1 or -1 as defined below:
- It returns 1 if the string is "sparrow" or "mynah"
- It returns 0 if the string is "penguin".
- It returns -1 if it is none of the above

Note: You can use the following C standard library function to compare two strings:

```
int strcmp(char s1[], char s2[]);
```

It compares the strings in the two arrays `s1` and `s2` and returns 0 only if they are identical, for example `strcmp(a,"hello")` will return 0 if the string in `a` is equal to "hello". `strcmp(a,"bye")`; will return a non-zero value if `a` has the string "hello".

6. Checking digits:

[5]

Write a function `kap()`. It takes one integer parameter n . If n has in its units place: 4, 9 or 5 then it prints "It is a kap" else it prints "It is not a kap".

Constraint: You must use a **switch-case** and NOT an **if-else**.

7. Searching for TC numbers

[2+8=10]

A TC number is one which can be written as the sum of two-cubes in at least two different ways (with positive integers). For example $152 = 3^3 + 5^3$ is not a TC number because there is no other way to write it as a sum of two cubes. On the other hand, $1729 = 12^3 + 1^3$ and $1729 = 10^3 + 9^3$, so 1729 is a TC number. This problem shows how to find if a number is a TC number.

(a) Write a function with prototype:

```
int checkprod(int a, int b, int t);
```

It tests if $a^3 + b^3$ equals t . If yes, it returns 1, if not it returns 0. Remember that C does not have an exponentiation operator, just repeated multiplies is the way you can do it.

(b) Write another function with prototype:

```
int checktc(int n);
```

it checks if n is a TC number and returns 1 or 0 to say it is or it is not a TC number. To do that it tests every possible (i, j) integer pairs where $i \leq j$. Here is how it works:

- Maintain a variable *count*, see below how it is used.
- Loop over each value of i from 1 to $n - 1$
- For a given i loop over each value of j from i to $n - 1$
- Call `checkprod()` with arguments i, j and n to check if $i^3 + j^3$ equals n
- If it is then increment *count*;
- After checking every such i and j pair and coming out of the two loops, see if *count* is greater than 1, if it is, then the function returns 1 else it returns 0.

Fundamentals of Computing and Programming

Midterm Examination – Solutions

(Original exam: September 21, 2023; max score 40, total marks 43, 2 hr time limit)

Question 1 ($4 \times 2 = 8$ marks)

(a) `printf("%d %c", 'e' - 'a', 'a' + 4);`

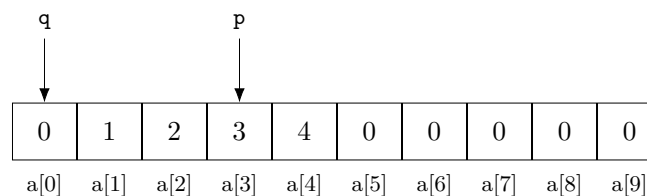
Output: 4 e

Explanation: character constants are just small integers (their character-code values) in C arithmetic. `'e' - 'a'` computes the integer difference between the codes of `'e'` and `'a'`, which is 4 (since `'e'` is the 5th letter and `'a'` the 1st), printed with `%d` as 4. `'a' + 4` computes the integer code of `'a'` plus 4, which is the code for `'e'`; printed with `%c` this displays the character `e`. Both expressions rely on the same underlying idea – letters are stored as consecutive integer codes – applied in opposite directions (subtracting two codes vs. adding an offset to one code).

(b) **Pointer/array diagram after `p = &a[3]; q = p - 3;`**

```
1 int a[10] = {0, 1, 2, 3, 4}; // a[5]..a[9] are implicitly 0
2 int *p, *q;
3 p = &a[3]; // p points to a[3]
4 q = p - 3; // pointer arithmetic: 3 ints "before" p
```

Since `p` points to `a[3]`, subtracting 3 (in pointer arithmetic, automatically scaled by `sizeof(int)` as covered in Chapter 4) moves `q` back to `a[0]` – *not* 3 bytes back, but 3 whole `int` elements back.



(c) `char name[20]="praggnananda"; printf("%s", name+6);`

Indexing `name` from 0: p(0) r(1) a(2) g(3) g(4) n(5) a(6) n(7) a(8) n(9) d(10) a(11). `name + 6` is a pointer to `name[6]`, so `%s` prints from index 6 onward until the null terminator.

Output: ananda

(d) `scanf("%s", name+6);` with input "chess", then `printf("%s", name)`

`scanf` writes the characters of "chess" (plus a null terminator) starting at index 6 of `name`, *overwriting* whatever was there, without touching indices 0–5:

Index	0	1	2	3	4	5	6	7	8	9	10	11
Before	p	r	a	g	g	n	a	n	a	n	d	a
After	p	r	a	g	g	n	c	h	e	s	s	\0

`printf("%s", name)` starts at index 0 and prints until it hits the (new, earlier) null terminator at index 11.

Output: praggchess

Teaching point: this question nicely shows that `%s` always reads from wherever the pointer starts, and the *original* tail of the string (**ananda**) is not “restored” or appended – it is simply overwritten and the new, shorter null-terminated string is all that `printf` ever sees.

Question 2: Downify (5 marks)

```

1 char downify(char a) {
2     if ( a >= 'A' && a <= 'Z' ) {           // __C__: is a an uppercase
3         return a - 'A' + 'a';               // __S__: shift to the
4     }                                       corresponding lowercase letter
5     return a;
6 }
```

Explanation: `a >= 'A' && a <= 'Z'` tests whether `a`'s character code falls in the uppercase-letter range. If so, `a - 'A'` gives its zero-based position within the alphabet (e.g. `'C' - 'A' = 2`), and adding `'a'` maps that same position into the lowercase range (e.g. `2 + 'a' = 'c'`). An equally valid alternative exploiting the fixed offset between corresponding upper/lowercase letters in ASCII is `return a + 32;`, but `a - 'A' + 'a'` is preferred since it does not depend on memorizing the numeric offset.

Question 3: Array Read/Print (5 marks)

```

1 #include <stdio.h>
2
3 int main(void) {
4     int a[10];
5
6     for (int i = 0; i < 5; i++) {
7         scanf("%d", &a[i]);
8     }
9
10    for (int i = 0; i < 5; i++) {
11        printf("%d\n", a[i]);
12    }
13
14    return 0;
15 }
```

Explanation: the array `a` is declared with capacity for 10 integers, but the question only asks to read and print 5 – the first loop fills `a[0]` through `a[4]`, and the second loop prints exactly those same five (not all ten, since `a[5]–a[9]` were never initialized and would contain indeterminate values).

Question 4: Array Sum (5 marks)

```
1 int sum(int a[], int n) {  
2     int total = 0;  
3     for (int i = 0; i < n; i++) {  
4         total += a[i];  
5     }  
6     return total;  
7 }
```

Explanation: `total` accumulates the running sum (initialized to 0, the identity element for addition), and the loop adds each of the `n` elements exactly once – a direct application of the accumulator-loop pattern from Chapter 2.

Question 5: Flying Birds (5 marks)

```
1 #include <string.h>  
2  
3 int fly(char s[]) {  
4     if (strcmp(s, "sparrow") == 0 || strcmp(s, "mynah") == 0) {  
5         return 1;  
6     } else if (strcmp(s, "penguin") == 0) {  
7         return 0;  
8     } else {  
9         return -1;  
10    }  
11 }
```

Explanation: `strcmp` returns 0 exactly when its two arguments are identical strings (Chapter 4), so `strcmp(s, "sparrow") == 0` tests whether `s` is "sparrow". The three-way `if/else if/else` chain directly mirrors the three cases specified in the question, with -1 as the catch-all default for any string that is none of the named birds.

Question 6: Checking Digits – `kap()` (5 marks)

```
1 void kap(int n) {  
2     switch (n % 10) {  
3         case 4:  
4         case 9:  
5         case 5:  
6             printf("It is a kap\n");  
7             break;  
8         default:  
9             printf("It is not a kap\n");  
10    }  
11 }
```

Explanation: `n % 10` extracts the units digit of `n` (assuming `n` is non-negative, which is the natural reading of the question). Stacking `case 4:`, `case 9:`, and `case 5:` with no `break` between them lets all three values share the same action (fall-through), printing "It is a kap"; any other units digit falls to `default`, printing "It is not a kap". This satisfies the constraint of using `switch-case` rather than `if-else`.

Robustness note (for discussion, not required for full marks): in C, `n % 10` can be *negative* when `n` is negative (e.g. `-14 % 10` is `-4`, not `6`), so a fully robust version might first take the absolute value of `n`, or of the computed remainder, before the `switch`:

```

1 void kap(int n) {
2     int units = n % 10;
3     if (units < 0) units = -units;    // normalize for negative n
4     switch (units) {
5         case 4:
6         case 9:
7         case 5:
8             printf("It is a kap\n");
9             break;
10        default:
11            printf("It is not a kap\n");
12    }
13 }

```

Question 7: Searching for TC Numbers ($2 + 8 = 10$ marks)

(a) `int checkprod(int a, int b, int t);` (2 marks)

```

1 int checkprod(int a, int b, int t) {
2     int cubeA = a * a * a;
3     int cubeB = b * b * b;
4     if (cubeA + cubeB == t) {
5         return 1;
6     }
7     return 0;
8 }

```

Explanation: since C has no exponentiation operator, a^3 and b^3 are each computed via two repeated multiplications ($a * a * a$), as the question specifies.

(b) `int checktc(int n);` (8 marks)

```

1 int checktc(int n) {
2     int count = 0;
3     for (int i = 1; i <= n - 1; i++) {
4         for (int j = i; j <= n - 1; j++) {
5             if (checkprod(i, j, n)) {
6                 count++;
7             }
8         }
9     }
10    if (count > 1) {
11        return 1;
12    }
13    return 0;
14 }

```

Explanation: the outer loop fixes i from 1 to $n - 1$; the inner loop fixes j from i to $n - 1$ (starting at i , not 1, so that each unordered pair (i, j) with $i \leq j$ is tested exactly once, avoiding double-counting (i, j) and (j, i) separately). Each pair is tested via `checkprod(i, j, n)`, incrementing `count` whenever $i^3 + j^3 = n$. If more than one such pair is found, n qualifies as a TC number.

Trace for $n = 1729$: the pairs found are $(1, 12)$, since $1^3 + 12^3 = 1 + 1728 = 1729$, and $(9, 10)$, since $9^3 + 10^3 = 729 + 1000 = 1729$. This gives `count = 2`, so `checktc(1729)` returns 1 – matching the example in the question.

Trace for $n = 152$: the only pair found is $(3, 5)$, since $3^3 + 5^3 = 27 + 125 = 152$; no other (i, j) pair with $i \leq j \leq 151$ sums to 152. This gives `count = 1`, so `checktc(152)` returns 0 – correctly identifying 152 as *not* a TC number, since it can be written as a sum of two cubes in only one way, matching the question’s own example.

BMATH FIRST YEAR, FUNDAMENTALS OF COMPUTING AND PROGRAMMING MIDERM EXAMINATION

Date: Friday, 13th September, 2024 | **Time:** 10 am - 12 noon

Part 1

Please answer all the questions. Each question carries 2 marks.

1. What would be the output of the following program.

```
#include <stdio.h>

int f(int a)
{
    int b = 0;
    while (a != 0)
    {
        b = b * 10 + a % 10;
        a = a / 10;
    }
    return b;
}

int main()
{
    int n = f(28567);
    printf("%d\n", n);
}
```

2. What does the following program do? Please explain in a single sentence.

```
#include <stdio.h>

int main (void)
{
    int n = 0;
    while (getchar() != EOF)
    {
        ++n;
    }
    printf("%d\n", n);
}
```

3. What will be the output of the following program.

```
#include <stdio.h>

int f(int a, int b)
{
    if (a == 0)
        return 0;
    else
        return b + f(a-1, b);
}

int main()
{
    int n = f(1234, 5);
    printf("%d\n", n);
}
```

4. What will be the output of the following program.

```
#include <stdio.h>

void swap(int x, int y)
{
    int tmp = x;
    x = y;
    y = tmp;
}

int main()
{
    int x = 10;
    int y = 20;

    swap(x, y);

    printf("%d %d\n", x, y);
}
```

5. What will be the output of the following program.

```
#include <stdio.h>

int main()
{
    int values[] = {1, 2, 3, 4, 5};
    int i;
    int sum = 0;

    for (i=0; i<5; i++)
    {
        sum += values[i];
        printf("%d\n", sum);
    }
}
```

Part 2

Please solve **any two** of the following three questions. Each question carries 5 marks.

6. If we list all the natural numbers below 10 that are multiples of 3 or 5, we get 3, 5, 6 and 9. Sum of these multiples is 23.

Write a program to compute the sum of all multiples of 3 or 5 below 1000.¹

(5 marks)

7. The sum of sequence of terms in the following series converges to $\frac{\pi}{8}$ very slowly.

$$\frac{1}{1.3} + \frac{1}{5.7} + \frac{1}{9.11} + \dots$$

For example, we take the first 3 terms:

$$\frac{1}{1.3} + \frac{1}{5.7} + \frac{1}{9.11} = \frac{1}{3} + \frac{1}{35} + \frac{1}{99} = 0.372005772005772$$

If we multiply that with 8, we'll get 2.976, which is quite far from π , but it gets closer as we add more and more terms.

Write a program to compute the value of π using 10000 terms in the above sequence.

(5 marks)

¹This problem is adopted from the Problem 1 of Project Euler. <https://projecteuler.net/problem=1>

8. Write a function `get_prime` that takes a positive integer n as argument and returns the n^{th} prime number. For example:

- `get_prime(1)` returns 2
- `get_prime(2)` returns 3
- `get_prime(3)` returns 5
- `get_prime(4)` returns 7

(5 marks)

Midterm Examination

Fundamentals of Computing and Programming
BMath First Year
Indian Statistical Institute, Bangalore Centre

Date: 12th September, 2025, Friday | Time: 2-5 pm

Total Marks: 35

Problem 1 - (4 marks)

What is the output of the following program?

```
#include<stdio.h>

int f(int a)
{
    int b = 0;
    while (a != 0)
    {
        b = b * 10 + a % 10;
        a = a / 10;
    }
    return b;
}

int main(void)
{
    int n = f(24589);
    printf("%d\n", n);
    return 0;
}
```

Problem 2 - (4 marks)

What is the output of the following program?

```
#include<stdio.h>

int f(int a, int b)
{
    return b == 0 ? 1 : a * f(a, b-1);
}

int main(void)
{
    printf("%d\n", f(2, 8));
    return 0;
}
```

Problem 3 - (4 marks)

What is the output of the following program? Explain your answer in a few lines.

```
#include<stdio.h>

void f(float u, int x, float y)
{
    x = (int) u;
    y = u - x;
    printf("%d %f\n", x, y);
}

int main(void)
{
    int x = 0;
    float y = 2.78;
    f(2.78, x, y);
    printf("%d %f\n", x, y);
    return 0;
}
```

Problem 4 - (4 marks)

What is the output of the following program? Explain your answer in a few lines.

```
#include<stdio.h>

void f(int a)
{
    if (a != 0)
    {
        f(a / 2);
        putchar('0' + a % 2);
    }
}

int main(void)
{
    f(11);
}
```

Problem 5 - (5 marks)

Write a program that asks the user to enter two integers, then calculates their greatest common divisor (GCD) using Euclid's Algorithm.

Problem 6 - (7 marks)

The sum of the primes below 10 is $2 + 3 + 5 + 7 = 17$. Write a program that finds the sum of all primes below 1,00,000.

Problem 7 - (7 marks)

Write a program that asks the user to enter a positive number, and it prints a table showing how many times each digit appears in the number.

BMath First Year – Fundamentals of Computing and Programming

Midterm Examination – Solutions

(Original exam: Indian Statistical Institute, Bangalore Centre;
12th September 2025, Friday, 2–5 pm; Total Marks: 35)

All outputs below were independently verified by compiling and running the exact code with `gcc`, not just hand-traced, to guarantee accuracy (particularly for the floating-point and large-sum questions).

Problem 1 (4 marks)

```
1 int f(int a) {  
2     int b = 0;  
3     while (a != 0) {  
4         b = b * 10 + a % 10;  
5         a = a / 10;  
6     }  
7     return b;  
8 }  
9 int main(void) {  
10     int n = f(24589);  
11     printf("%d\n", n);  
12     return 0;  
13 }
```

Output: 98542

Explanation: exactly as in the recurring "digit reversal" pattern from previous exams, each iteration peels the last digit off `a` (`a % 10`) and appends it as the new last digit of `b` (`b * 10 + digit`), while `a` shrinks via integer division. Tracing: `24589` → `b=9`, `2458` → `b=98`, `245` → `b=985`, `24` → `b=9854`, `2` → `b=98542`, then `a=0` and the loop stops – the digits of `24589` reversed give `98542`.

Problem 2 (4 marks)

```
1 int f(int a, int b) {  
2     return b == 0 ? 1 : a * f(a, b - 1);  
3 }  
4 int main(void) {  
5     printf("%d\n", f(2, 8));  
6     return 0;  
7 }
```

Output: 256

Explanation: this is linear recursion (Chapter 3) computing a^b : the base case $b=0$ returns 1 (the multiplicative identity), and each recursive step multiplies by one more factor of a while decrementing b , unwinding to $a \times a \times \dots \times a$ (b times). Here $f(2, 8) = 2^8 = 256$.

Problem 3 (4 marks)

```

1 void f(float u, int x, float y) {
2     x = (int) u;
3     y = u - x;
4     printf("%d %f\n", x, y);
5 }
6 int main(void) {
7     int x = 0;
8     float y = 2.78;
9     f(2.78, x, y);
10    printf("%d %f\n", x, y);
11    return 0;
12 }

```

Output:

```

1 2 0.780000
2 0 2.780000

```

Explanation: `f`'s parameters `u`, `x`, `y` are entirely separate, freshly-initialized copies of whatever was passed in – C's pass-by-value semantics (Chapter 3) – so any assignment to them inside `f` has *no effect whatsoever* on `main`'s `x` and `y`. Inside `f`: `u = 2.78`, `(int) u` truncates to 2, so `x` becomes 2; `y = u - x = 2.78 - 2 = 0.78`, printed by `f`'s own `printf` as `2 0.780000`. Control then returns to `main`, whose own `x` and `y` were never touched – they still hold their original values 0 and 2.78, printed as `0 2.780000`. This question is an excellent one for reinforcing that pass-by-value copies extend to *every* parameter type (`int` and `float` alike), not just to the specific pattern seen in earlier `swap` examples.

Problem 4 (4 marks)

```

1 void f(int a) {
2     if (a != 0) {
3         f(a / 2);
4         putchar('0' + a % 2);
5     }
6 }
7 int main(void) {
8     f(11);
9 }

```

Output: 1011 (*printed with no trailing newline, since the code never calls `putchar('\n')` or similar*)

Explanation: this function prints the binary representation of `a`. The key idea is that the recursive call `f(a/2)` happens *before* `putchar`, so all the higher-order recursive calls (and their eventual `putchars`) complete *first*, before the current call's own digit is printed – meaning digits are emitted from most-significant to least-significant, in the correct left-to-right order, with no need to reverse anything afterward.

Trace for `f(11)` (calls happen top to bottom; `putchars` execute bottom to top, i.e. in reverse call order):

Call	Action
$f(11)$	calls $f(5)$, then will <code>putchar('0'+11%2) = '1'</code>
$f(5)$	calls $f(2)$, then will <code>putchar('0'+5%2) = '1'</code>
$f(2)$	calls $f(1)$, then will <code>putchar('0'+2%2) = '0'</code>
$f(1)$	calls $f(0)$, then will <code>putchar('0'+1%2) = '1'</code>
$f(0)$	$a = 0$, base case: does nothing, returns immediately

Unwinding back up: $f(1)$ prints '1', then $f(2)$ prints '0', then $f(5)$ prints '1', then $f(11)$ prints '1' – giving the sequence 1 0 1 1, i.e. 1011, which is indeed 11 in binary ($8+2+1 = 11$).

Problem 5 (5 marks): GCD via Euclid's Algorithm

```

1  #include <stdio.h>
2
3  int main(void) {
4      int a, b;
5      printf("Enter two integers: ");
6      scanf("%d %d", &a, &b);
7
8      int x = a, y = b;
9      while (y != 0) {
10         int r = x % y;
11         x = y;
12         y = r;
13     }
14     printf("GCD = %d\n", x);
15     return 0;
16 }
```

Explanation: Euclid's algorithm rests on the identity $\text{gcd}(x, y) = \text{gcd}(y, x \bmod y)$, repeated until the second value becomes 0, at which point the first value is the GCD. Each loop iteration replaces (x, y) with $(y, x \bmod y)$; since y strictly decreases (and stays non-negative) each step, the loop is guaranteed to terminate.

Verified test runs: $\text{gcd}(48, 18) = 6$; $\text{gcd}(1071, 462) = 21$ – both confirmed by compiling and running the program above.

Problem 6 (7 marks): Sum of Primes Below 1,00,000

```

1  #include <stdio.h>
2  #define LIMIT 100000
3
4  int main(void) {
5      static int isComposite[LIMIT] = {0};    // static: avoids a large
6      long sum = 0;                           stack allocation
7
8      for (int i = 2; i < LIMIT; i++) {
9          if (!isComposite[i]) {               // i is prime
10             sum += i;
11             for (long j = (long)i * i; j < LIMIT; j += i) {
12                 isComposite[j] = 1;           // mark multiples of i as
13             }                                 composite
14     }
```

```

14     }
15 }
16 printf("%ld\n", sum);
17 return 0;
18 }

```

Explanation: this is the Sieve of Eratosthenes. `isComposite[i]` starts `false` (0) for every i ; whenever the outer loop reaches an i that is still marked prime, it is added to `sum`, and *all* its multiples starting from i^2 (smaller multiples of i were already marked by smaller prime factors) are marked composite so they are skipped later. This is dramatically faster than testing each number individually by trial division, which matters here since the limit is 100000. The array is declared `static` so it is allocated once in a fixed data segment rather than risking a stack overflow from a 100000-element local array.

Verified output (via compilation): 454396537.

Sanity check against the question's own worked example: summing primes below 10 by hand gives $2 + 3 + 5 + 7 = 17$, matching the question statement – confirming the same algorithm, only with `LIMIT` changed from 10 to 100000, is correct.

Problem 7 (7 marks): Digit Frequency Table

```

1  #include <stdio.h>
2
3  int main(void) {
4      long n;
5      int digitCount[10] = {0};
6
7      printf("Enter a positive number: ");
8      scanf("%ld", &n);
9
10     long temp = n;
11     if (temp == 0) {
12         digitCount[0] = 1;
13     }
14     while (temp > 0) {
15         int d = temp % 10;
16         digitCount[d]++;
17         temp = temp / 10;
18     }
19
20     printf("Digit Count\n");
21     for (int d = 0; d <= 9; d++) {
22         printf("  %d      %d\n", d, digitCount[d]);
23     }
24     return 0;
25 }

```

Explanation: `digitCount` is a 10-element array acting as a frequency table, one slot per possible digit 0–9 (Chapter 2's array material). The `while` loop repeatedly extracts the last digit of `temp` (`temp % 10`), increments that digit's counter, and removes the digit (`temp / 10`) – the same digit-extraction idiom used in Problem 1 – continuing until every digit of the original number has been counted. The special-case check for `n == 0` ensures the single digit 0 is still counted correctly, since the main loop's condition `temp > 0` would otherwise never execute for an input of 0.

Verified test run (input 112233458):

Digit	Count
0	0
1	2
2	2
3	2
4	1
5	1
6	0
7	0
8	1
9	0

This matches the input exactly: two each of 1, 2, 3, and one each of 4, 5, 8.

